Architecture of the Grassroots Networking Transport

This chapter documents the `tum-implementation` *branch. Every factual claim is grounded in the source; file/line references are approximate as the code evolves.*

Contents

Chapter 1

Architecture of the Grassroots Networking Transport

1.1 Introduction, Scope & Design Philosophy

Grassroots Networking is a *peer-to-peer messaging transport*: a thin layer whose sole responsibility is to move opaque payloads between devices, addressed by cryptographic identity, over two very different media. It is deliberately *not* an application. There is no notion of a chat room, a contact list, a conversation, or a user interface anywhere in its contract; those concepts belong to the applications that sit on top of it. The system's own `pubspec.yaml` describes the package (`grassroots_networking`, version `0.1.0`) as a transport for GSG, the Grassroots Social-Graph application, and the formal specification frames the whole effort as a networking API meant to replace the madGLP `IsolateManager` so that GLP agents can run on separate physical devices and communicate directly, peer-to-peer, over a medium-agnostic substrate (`docs/GLP_Networking_API/sections/introduction.tex`, `docs/GLP_Networking_API/sections/api.tex`). Everything in this chapter should be read through that lens: the layer is plumbing, and its correctness is judged by whether packets reach the right identity, sealed, over whichever medium is available.

1.1.1 Design Philosophy

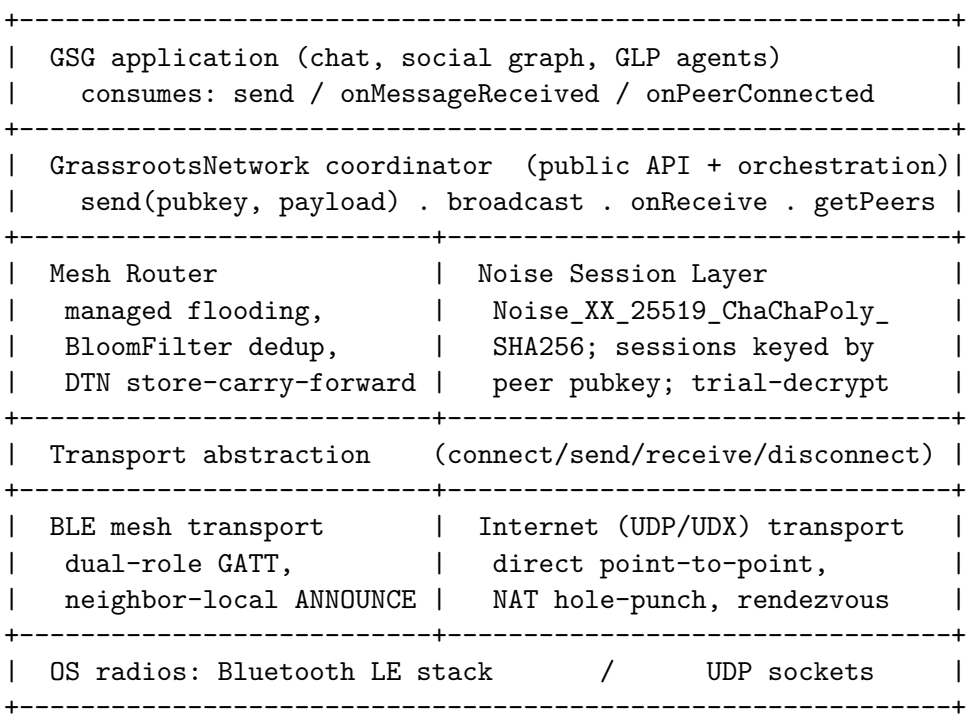
Four principles, drawn from the project's engineering charter and the GLP specification, shape every decision that follows.

- **Opportunistic mesh delivery.** Over Bluetooth Low Energy (BLE) the transport is a multi-hop *opportunistic mesh*: every node relays packets toward the recipient by TTL-bounded, deduplicated *managed flooding*, and nodes *store-carry-forward* — caching packets for recipients that are momentarily out of range and re-flooding them when the recipient reappears. This lets a user reach a friend who is not a direct neighbor. The Internet transport, by contrast, stays *direct* point-to-point. Crucially, relaying is not friend-gated and not authenticated hop-by-hop: a relay forwards sealed bytes it cannot read, addressed only by recipient identity.
- **Identity is a key pair.** Every device holds an Ed25519 key pair, and the 32-byte public key *is* the peer's globally unique identity, used for addressing, authentication, and signing, persistent across sessions (`docs/GLP_Networking_API/sections/api.tex`). Nicknames are cosmetic; all trust flows from cryptographic verification. This is why `send` rejects any recipient key that is not exactly 32 bytes (`lib/src/grassroots_network.dart:1877`).

- **Two transports, one interface.** BLE covers nearby peers with no Internet; the Internet transport (UDP via UDX, with NAT hole-punching) covers the globe. Both surface the same abstraction to the coordinator — connect, send, receive, disconnect — and BLE is preferred when both are usable (`lib/src/grassroots_network.dart:1943`). The two transports are strictly independent: losing or disabling one has zero effect on the other's reachability state.
- **Clean breaks, not compatibility shims.** Because there are no deployed installations in the wild, the project renames, restructures, and breaks wire formats whenever doing so improves the design, rather than carrying legacy decoders or dead code. This principle is not rhetorical — the wire format was recently and deliberately broken (see below), and decoders are written to *reject* malformed or truncated payloads rather than tolerate a hypothetical older version.

1.1.2 A Corrected Layered Model

The system is organized as a stack. An application such as GSG never touches radios directly; it calls the `GrassrootsNetwork` coordinator, which owns the public API and mediates every layer beneath it.



The coordinator (`lib/src/grassroots_network.dart`) wires together a `ProtocolHandler`, a `FragmentHandler`, a `NoiseSessionManager`, a `MessageRouter`, and a `SignalingService`, and exposes the specified surface: `initialize`/`start`/`stop`, `send`, `broadcast`, `getPeers`, `isPeerReachable`, and the callbacks `onPeerDiscovered`, `onPeerConnected`, `onMessageReceived` (`lib/src/grassroots_network.dart:578-633`, `757`, `1872`). Two facts about the layering are load-bearing throughout the chapter. First, the mesh router and the Noise session layer are peers, not one atop the other: the router dispatches packets by their *outer* type and, for a sealed packet addressed to us, asks the session layer to `trialDecrypt` it against every active session — the AEAD tag, not any header field, identifies the sender (`lib/src/grassroots_network.dart:3885`). Second, sessions are keyed by peer public key, not by transport path, so a Noise session survives a change of relay path or even the loss of a whole transport; `stop` tears down transports but deliberately keeps sessions (`lib/src/grassroots_network.dart:963-992`).

1.1.3 System Context

An outbound message is sealed to the recipient's Noise session and, on BLE, flooded to all neighbors; success is a positive flood count, and if no transport is usable the message is queued in a per-recipient FIFO to be re-flooded when conditions change (`lib/src/grassroots_network.dart:1905-1967`). Inbound, a peer is only reported *connected* once an authenticated Noise session exists — reachability is gated on cryptographic authentication, not on a mere radio contact (`lib/src/grassroots_network.dart:2716-2733`). Presence and identity ride on a separate, non-relayed channel: the self-signed ANNOUNCE, a neighbor-local broadcast (TTL 1) whose payload carries the full public key, nickname, and an Ed25519 signature over that body (`lib/src/grassroots_network.dart:4928-4933`). ANNOUNCE is the *only* thing signed on the wire; all message, ack, read-receipt, and signaling traffic authenticates end-to-end by decrypting under a verified Noise session, never by a per-packet signature.

1.1.4 Roadmap

The remainder of this chapter builds outward from these foundations. *Identity, Trust & the Security Model* develops the key-pair identity, the sender-anonymous envelope, and why relaying is unauthenticated by construction. *Wire Format: Packets & the Data Model* specifies the 58-byte header and the sealed `SecureFrame` that now carries content type and fragmentation inside the ciphertext. *The Transport Abstraction & the BLE Mesh Transport* covers the common interface and the mandatory dual-role BLE design. *Mesh Routing* details managed flooding, the `BloomFilter` dedup, and the DTN store-carry-forward cache. *The Noise Session Layer* covers the `Noise_XX` handshake, trial-decrypt, and the AAD construction. *State Management* explains the Redux projection, and later sections walk through end-to-end interactions and close with design decisions, trade-offs, and future work.

1.1.5 Lineage and a Warning about Stale Documentation

The transport descends from the Bitchat protocol, and that lineage is still visible in the repository's `README.md`, which is titled "Bitchat Transport for GSG" and describes classes and dependencies that no longer exist (`README.md:1-7`). **This README is stale and must be treated as history only.** It contradicts the current design on nearly every load-bearing number — most dangerously, it documents a 154-byte packet header carrying a 32-byte sender field and a 64-byte per-packet signature (`README.md:173-183`). The real header is 58 bytes, sender-anonymous, and unsigned (only ANNOUNCE is signed at the payload level); this chapter relies exclusively on the verified source, not the README. Two further scoping notes anticipate common misreadings: rendezvous servers are the *current* NAT-bootstrap mechanism and remain fully implemented (their replacement by well-connected friends and signed invite links is a design note, hence future work), and the multi-hop Noise handshake between peers who have never been in direct range is likewise future work — handshakes today are neighbor-local (TTL 1). Where this chapter describes a mechanism, it describes what the code does; where it labels something future work, that work is not yet shipped.

1.2 Identity, Trust & the Security Model

The security model of the Grassroots transport is built on a single, deliberately austere primitive: a device's cryptographic key pair *is* its identity. Everything else — trust decisions, session authentication, sender anonymity, and the two distinct trust boundaries described below — is derived from that key pair rather than from any server-issued credential, account, or transport-

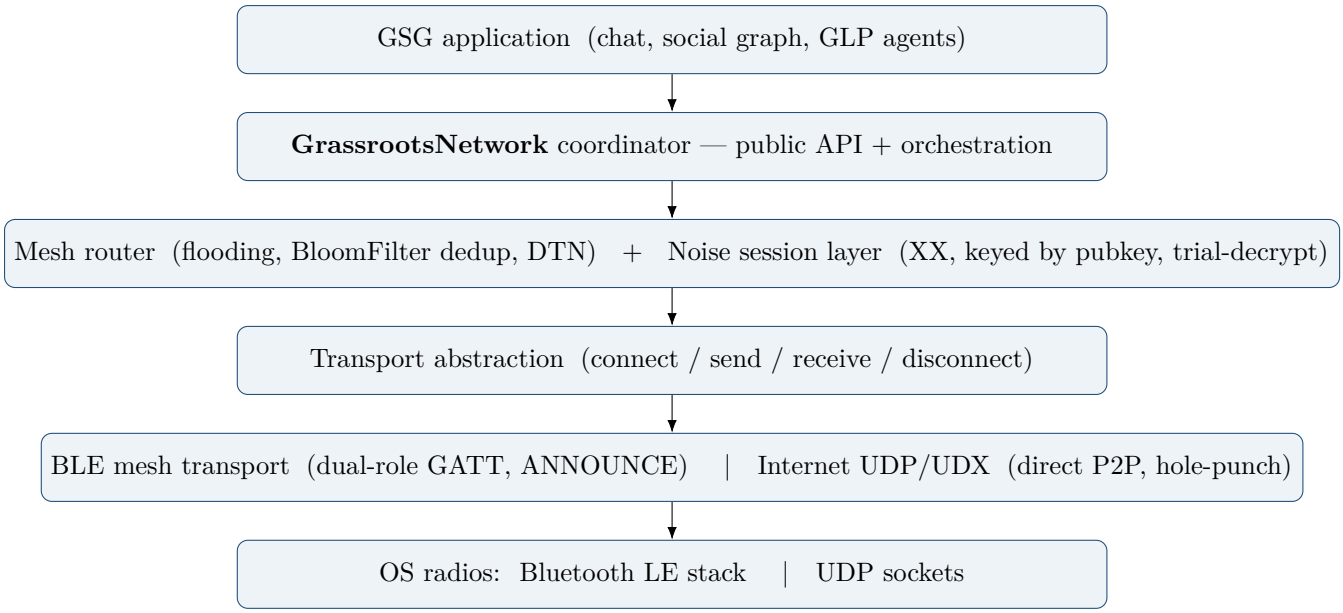


Figure 1.1: The layered architecture. The coordinator owns the public API; the mesh router and Noise session layer sit beneath it as peers; two transports share one abstraction.

level address. This section explains what that identity is, how it authenticates a Noise session, how the sender-anonymous mesh envelope constrains what an adversary can learn, and how first contact is gated. It closes with the threat model and the clearly-labelled future redesign of the handshake.

1.2.1 Cryptographic identity

Every device holds an Ed25519 key pair, generated once on first launch (`GrassrootsIdentity.generate`) and persisted by the application layer. The 32-byte Ed25519 public key is the peer's canonical identity; the 64-byte private key (32-byte seed concatenated with the public key, per `lib/src/models/identity.dart`) never leaves the device. It serves two purposes: signing ANNOUNCE bodies, and deriving the peer's Noise X25519 static key via the birational map (libsodium's `skToCurve25519`, `noise_session_manager.dart:299`) — the derivation on which every Noise session in the next subsection rests. Nicknames are purely cosmetic — a mutable display string carried in ANNOUNCE — and carry no authority; two peers with the same nickname are still distinct identities, and a mismatched nickname never overrides the key-based identity.

The public key also seeds BLE discovery. Each device advertises a service UUID whose *fixed* 8-byte Grassroots prefix (the first 8 bytes of `SHA-256("grassroots")`) marks it as a Grassroots device, followed by an 8-byte suffix that rotates every 15 minutes (`deriveServiceUuidForSlot`: the first 8 bytes of `SHA-256("grassroots ble suffix" | pubkey | slot)`). The rotating suffix is an *unlinkable* recognition hint — a peer who knows the key recomputes the current and adjacent slots' UUIDs to recognise the device and pre-filter scans, while a third party who does not know the key sees a beacon that changes every slot and cannot track it — but it is explicitly *not* an authorization proof: it is only a truncated hash, and the full public key is authenticated separately by the signed ANNOUNCE.

1.2.2 Noise sessions and the birational static-key check

End-to-end confidentiality and authentication run over Noise, pattern `Noise_XX_25519_ChaChaPoly_SHA256` (`lib/src/session/noise_session_manager.dart`). A crucial design decision links the two key systems: the Noise *static* key is not an independent secret but the X25519 form of the Ed25519 identity, derived via the standard birational map (libsodium's `crypto_sign_ed25519_sk_to_curve25519` / `pk_to_curve25519`). Because the public half of the static is a deterministic public function of the Ed25519 public key, any peer that knows a counterpart's identity can *recompute* the static it should present.

This is exactly what `_verifyRemoteStatic` does. During the XX handshake the remote's static key is delivered encrypted inside messages 2 and 3; on receiving it, the local side recomputes the expected X25519 static from the peer's claimed Ed25519 identity and compares (in constant time) against what Noise delivered. A peer that presents a static not belonging to the identity it claims — or a tampered transcript — fails the check, and the handshake is aborted and the session entry discarded. Handshake authenticity therefore rests on the Noise transcript *plus* this static binding; it is what ties the ephemeral Noise session to a durable, verifiable Ed25519 identity. Sessions are keyed by the peer's Ed25519 public key, never by transport path, so one session serves both the direct UDP path and any relayed BLE path and survives changing relay routes.

1.2.3 The sender-anonymous envelope and its consequences

The outer packet header is 58 bytes and carries only the *recipient* public key (32 bytes), plus type, TTL, timestamp, packet ID, and payload length (`lib/src/models/packet.dart`). There is no sender field and no whole-packet signature. This single structural choice cascades through the whole security model:

- **Only ANNOUNCE is signed.** The wire `PacketType` enum has exactly three values — `announce`, `noiseHandshake`, and `secure`. ANNOUNCE is the one packet whose sender is *meant* to be visible: it is a neighbour-local, non-relayed presence broadcast whose payload ends with an Ed25519 signature over the body (pubkey, version, nickname, address candidates), verified hop-locally in `decodeAnnounce` against the embedded pubkey. Every other packet carries no wire signature at all.
- **Authentication is end-to-end, inside Noise.** A `secure` packet is an opaque, session-sealed envelope. Its logical content type (message, ack, read receipt, or signaling) and any fragmentation live *inside* the encrypted payload as a `SecureFrame` (`[contentType][fragIndex][fragCount][messageId][chunk]`), so a relay never learns the message class — it sees only a `secure` blob addressed by recipient. The recipient trusts the content solely because it decrypts under a Noise session whose static was verified above; the ChaCha20-Poly1305 AEAD tag is the authentication.
- **Trial-decrypt demultiplexing.** Because the envelope names no sender, the recipient cannot look up the right session by header. Instead `trialDecrypt` attempts decryption against every active session in turn; the AEAD tag identifies the correct one (a wrong session simply fails the tag and is skipped), and the matching session's peer key is returned as the recovered sender.
- **AAD excludes mutable fields.** The application AAD (`_applicationAad`) binds the constant `secure` type byte, the sender pubkey, the recipient pubkey, and the 16-byte packet ID — but deliberately *excludes* TTL and timestamp, which every relay mutates and therefore cannot be authenticated end-to-end. The content type is nonetheless authenticated, because it sits inside the AEAD-protected plaintext.

- **Bounded replay window.** Each transport session tracks received 64-bit nonces and rejects a repeat as a replay, retaining the most recent 2048 nonces (`_rememberReceivedNonce`) — a bounded window that resists replay without unbounded memory growth.

1.2.4 Cold-call trust and two trust boundaries

First contact from a stranger is governed by a `ColdCallTrustLevel` setting (`lib/src/store/settings_state.dart`), which defaults to `closed`. In `open` mode a device completes the BLE ANNOUNCE exchange with unknown nearby peers, permitting cold-call first contact; in `closed` mode it does not complete ANNOUNCE with unknown peers, and friend-only metadata is withheld until a verified ANNOUNCE has authenticated an accepted friend.

Two distinct trust boundaries coexist, and it is important not to conflate them:

- **Open BLE relay.** The mesh relays `secure` packets for *arbitrary* recipients — a relay forwards sealed, recipient-addressed bytes for peers it has no relationship with. It can do so safely precisely because it learns neither the sender nor the content; relaying is unverified by construction, bounded instead by TTL, dedup, and rate limits (discussed in the mesh routing section).
- **Friend-only UDP signaling.** Internet hole-punch coordination is, by contrast, strictly friend-gated. A device only registers friends' addresses, only answers address queries for friends, and only dispatches `PUNCH_INITIATE` to friends: `signaling_service.dart` guards these paths with explicit `isFriend` / `getFriendship` checks (e.g. it refuses to mediate when `targetPeer == null || !targetPeer.isFriend`). This boundary exists because signaling exposes addresses — real metadata — whereas the BLE mesh only ever forwards sealed content.

1.2.5 Threat model

Against a relay or passive BLE observer, the design confines the leak to routing metadata. Such an adversary learns the *recipient* of a `secure` packet, its size, TTL, timestamp, and packet ID, and can therefore perform traffic-flow and dedup analysis. It does *not* learn the sender (absent from the envelope), the message content, or even the message *class* — message, ack, receipt, and signaling are indistinguishable once sealed. It cannot forge or replay content: forgery fails the AEAD tag under a session it does not hold, and replay is caught by the 2048-nonce window. ANNOUNCE, being signed, cannot be spoofed to impersonate an identity. The residual disclosures — recipient visibility and the fact that ANNOUNCE reveals presence and identity to neighbours — are the accepted, deliberate cost of an open, unverified relay; they are the price of a mesh that reaches beyond direct neighbours without hop-by-hop signatures.

1.2.6 Future work: KK/IK handshake redesign

The current handshake is the interactive Noise XX pattern and is *neighbour-local*: handshake packets are not flooded (TTL 1), so two peers must be in direct BLE range (or on a direct UDP path) to establish a session. A from-scratch multi-hop handshake using the pre-shared-static Noise patterns *IK* and *KK* — which would let a session be negotiated across relays to a peer whose identity is already known — is *not* implemented and is planned future work, to be accompanied by a security audit of the redesigned handshake. Until then, session establishment remains a direct, neighbour-local step even though the resulting session is fully path-independent and survives relaying.

—

Note to the orchestrator: the grounding files were passed with a literal ‘undefined/‘ path prefix and do not exist on disk, so I verified every claim directly against the source. Authoritative files consulted (all absolute): ‘/Users/dbachar/git/technion/grassroots_networking/.claude/worktrees/zealous-noether-ff8f0f/lib/src/session/noise_session_manager.dart‘, ‘.../lib/src/models/packet.dart‘, ‘.../lib/src/models/identity.dart‘, ‘.../lib/src/models/secure_frame.dart‘, ‘.../lib/src/protocol/protocol_handler.dart‘, ‘.../lib/src/store/settings_state.dart‘, ‘.../lib/src/signaling/signaling_service.dart‘. All numbers used (58-byte header, three packet types, 2048-nonce window, XX pattern, birational static check, friend-only signaling) are confirmed in those files.

1.3 Wire Format, Packets & the Data Model

At the base of the transport sits a single on-wire container: the `GrassrootsPacket`. Every byte that crosses a Bluetooth or Internet link is one such packet, and its design encodes the transport's central privacy commitment directly into the header. The header is exactly **58 bytes** (`GrassrootsPacket.headerSize = 58`, `lib/src/models/packet.dart`) — not the 154-byte, sender-carrying, whole-packet-signed header that the stale in-repo README still advertises. The difference is not cosmetic: the current header is *sender-anonymous*. It carries the recipient and nothing that identifies the originator.

The 58-byte header, field by field

The header serialises the following fixed layout, with all multi-byte integers big-endian:

```
[0]      type        (1 byte)   0x01 announce | 0x02 noiseHandshake | 0x03 secure
[1]      ttl         (1 byte)   decremented per relay hop, dropped at 0
[2-5]    timestamp   (4 bytes)  seconds since epoch
[6-37]   recipient   (32 bytes) recipient public key; all-zero = broadcast
[38-53]  packetId    (16 bytes) UUID, used for dedup / loop suppression
[54-57]  length      (4 bytes)  payload length
[58..]   payload     (variable) Noise-sealed for secure packets
-------
58 bytes total header
```

Two absences are the whole point. There is *no sender field* and *no whole-packet signature*. A relay can route a packet toward its `recipient` without ever learning who sent it, and — because there is no signature over the header — a relay also *cannot* authenticate the packet it forwards. This is deliberate: hop-by-hop authentication is traded away for sender-anonymity, and the resulting open, unverified relay is instead bounded by `ttl` (default 7), by `packetId` deduplication, and by per-neighbour rate limits (see the mesh-routing discussion elsewhere in this chapter). The `timestamp` and `recipient` round out the header; an all-zero recipient decodes to the broadcast sentinel. The `length` field at offset 54 doubles as the stream framer: a byte-stream transport such as UDX accumulates bytes until `headerSize + length` are present before it treats the buffer as one packet (`peekPayloadLength`).

Three wire types, and why the fourth field moved inside

The wire `PacketType` enum has been deliberately tightened to exactly **three** values: `announce` (`0x01`), `noiseHandshake` (`0x02`), and `secure` (`0x03`). Everything that is not a presence broadcast

or a handshake message is a single, opaque `secure` envelope. This is a recent, shipped refactor (client test suite green at 587/587; the `bootstrap_anchor` mirror is a pending follow-up and still speaks the older multi-type wire until it is updated).

The motivation is a metadata leak. The previous wire distinguished `secureMessage`, `secureAck`, `secureSignaling`, and several `secureFragment*` types directly in the header — so a relay, which can read the header but never the sealed body, could still tell an acknowledgment from a message from a signaling packet, and could see fragment boundaries. Collapsing all of these to one `secure` type closes that leak: a relay now sees only `0x03` for *all* sealed traffic and can distinguish neither the message class nor the fragment structure. This is the same class of leak the handshake security audit flagged.

The content type and fragmentation information did not disappear; they *moved inside* the sealed payload, into a `SecureFrame` (`lib/src/models/secure_frame.dart`). The `SecureFrame` is the plaintext that gets sealed into a `secure` packet, with a 21-byte inner header:

```
[0]      contentType (1 byte)   0x01 message | 0x02 ack | 0x03 readReceipt | 0x04 signaling
[1-2]    fragIndex   (2 bytes)  0-based fragment index
[3-4]    fragCount   (2 bytes)  total fragments; 1 = whole message
[5-20]   messageId   (16 bytes) logical message id (binary UUID)
[21..]   chunk       (variable) this fragment's bytes
```

Because `contentType` lives inside the AEAD-protected plaintext, it is authenticated end-to-end for free — the recipient learns the true content class only after a successful decrypt, and no one in between can. The `nack` type present in the old enum was removed as dead. Fragmentation is now framing metadata (`fragIndex`/`fragCount`) rather than a family of wire packet types: a non-fragmented message is simply `fragCount == 1`.

Encode/decode discipline: throw, never tolerate

The codecs follow the project's no-compatibility-shim rule strictly. `GrassrootsPacket.deserialize` throws a `FormatException` if the buffer is shorter than the 58-byte header or if the declared payload is truncated; `PacketType.fromValue` and `ContentType.fromValue` throw on any unknown tag rather than falling back; a recipient key that is not exactly 32 bytes throws in the constructor; and `SecureFrame.decode` throws when the data is shorter than its 21-byte header. There is no "gracefully handle an older, shorter payload" branch anywhere — since no prior wire version exists in the wild, a malformed payload is malformed, and the decoder says so.

ANNOUNCE: the one signed, non-relayed packet

`ANNOUNCE` is the single exception to sender-anonymity, because presence is precisely a statement about identity. It is neighbour-local (TTL 1, never flooded) and carries a payload-level Ed25519 self-signature. Its payload layout (`lib/src/protocol/protocol_handler.dart`) is:

```
pubkey(32) + version(2) + nickLen(1) + nick + candidateCount(2)
           + repeated[ candidateLen(2) + candidate ] + signature(64)
```

`decodeAnnounce` strips the trailing 64-byte signature, verifies it over the body against the *embedded* public key, and throws `ANNOUNCE signature invalid` on failure; it also rejects payloads below the 96-byte (32+64) floor and any truncated candidate field. This is the only place the transport verifies a signature on the wire; message, ack, read-receipt, and signaling content authenticate purely end-to-end, by trial-decryption against Noise sessions (discussed in the section on the Noise session layer).

Fragmentation and reassembly as SecureFrames

Large payloads are split by the `FragmentHandler` (`lib/src/protocol/fragment_handler.dart`). A payload longer than the `fragmentThreshold` of **320 bytes** is chunked into `SecureFrame`s of at most `maxFragmentPayload = 270 bytes` each, sharing one `messageId` and carrying ascending `fragIndex` out of a common `fragCount`; `framesFor(...)` produces the list and the sender floods them `fragmentDelay = 20 ms` apart. Crucially, each fragment is an ordinary `secure` packet — reassembly happens *after* decryption, keyed by the inner `messageId`, so relays never see fragment structure. `accept(frame)` buffers fragments and returns the complete payload once every index has arrived (immediately for a single-fragment message), with a stale-reassembly timeout of two minutes. This per-message fragment budget is distinct from `maxPayloadSize = 442` (computed as $500 - 58$), the soft single-packet ceiling chosen to keep one sealed packet under the roughly 500-byte BLE MTU; the two budgets are maintained independently. Note also that relay/loop dedup is keyed on the *outer* `packetId`, whereas delivery dedup is keyed on the *inner* `messageId` — the two identifiers serve different layers.

The application data model: Blocks

Above the transport, an application payload is a serialised `Block` (`lib/src/models/block.dart`) — the unit GSG builds its blocklace from. A `Block` is a one-byte `type` tag followed by its body, and `Block.deserialize` reads `data[0]` as the type and treats the remainder as the payload. The current `BlockType`s are `say` (`0x01`), `friendshipOffer` (`0x02`), `friendshipAccept` (`0x03`), and `friendshipRevoke` (`0x05`); value `0x04` is a reserved gap. A `SayBlock` adds a one-byte `subtype`, so its wire form is `[0x01 type][subtype][body]`. The two subtypes make picture messaging *first-class content*: `text` (`0x00`) carries raw UTF-8, while `picture` (`0x01`) carries `[viewOnce:1][mimeLen:1][mime][imageBytes]` — the `viewOnce` flag drives the "render blurred, delete on first view" behaviour, and images are compressed for BLE and stored content-addressed by SHA-256 (`lib/src/services/media_service.dart`). No `Block` subtype carries a signature; signing lives only in ANNOUNCE, consistent with the model above.

From app payload to sealed bytes

The end-to-end path is therefore a clean stack of nested framings. An application constructs a `Block` and serialises it. That byte string becomes the `chunk` of one or more `SecureFrame`s (one if it is within the 320-byte threshold, several otherwise), each tagged `contentType = message` and carrying a shared `messageId`. Each `SecureFrame` is encoded and then *sealed* by the Noise session keyed to the recipient, producing the opaque payload of a `secure` (`0x03`) `GrassrootsPacket` addressed only to that recipient. What finally travels the mesh is a 58-byte anonymous header wrapping ciphertext — the message class, the fragment structure, the sender, and the content all hidden inside the seal, visible only to the two endpoints.

Sources verified against current source: `lib/src/models/packet.dart`, `lib/src/models/secure_frame.dart`, `lib/src/protocol/protocol_handler.dart`, `lib/src/protocol/fragment_handler.dart`, `lib/src/models/block.dart`, `lib/src/services/media_service.dart`.

1.4 Transport Abstraction & the BLE Mesh Transport

Grassroots exposes exactly one abstraction to the layers above it, and hides two very different radios beneath it. A message router does not know whether the bytes it hands down will travel over Bluetooth to a device in the same room or over the Internet to a device on another

Table 1.1: The 58-byte cleartext wire header (sender-anonymous) and the 21-byte inner `SecureFrame` that lives inside the sealed payload.

Outer wire header — 58 bytes, cleartext, sender-anonymous		
Offset	Bytes	Field
0	1	packet type: announce / noiseHandshake / secure
1	1	TTL (decremented at each relay hop)
2	4	timestamp
6	32	recipient public key (zeros = broadcast)
38	16	packet id (dedup / loop prevention)
54	4	payload length
Inner SecureFrame — 21-byte header, sealed inside the payload		
0	1	content type: message / ack / readReceipt / signaling
1	2	fragment index
3	2	fragment count (1 = not fragmented)
5	16	message id (logical id: reassembly + ACK correlation)
21	n	chunk (content bytes; the whole payload when fragCount = 1)

continent; it only knows the `TransportService` interface. This section describes that interface and its lifecycle, then examines the Bluetooth Low Energy (BLE) transport in depth — discovery, dual-role convergence, and how packets actually move over GATT.

1.4.1 The Unified Transport Interface

Every transport implements the abstract `TransportService` class (`lib/src/transport/transport_service.dart`). The interface is deliberately small: type and display metadata, a `state` getter, two event streams (`dataStream` for inbound bytes, `connectionStream` for connect/disconnect events), `connectedCount`/`isActive` getters, the lifecycle verbs `initialize()`, `start()`, `stop()`, and `dispose()`, and the two send primitives `sendToPeer(peerId, data)` and `broadcast(data, {excludePeerIds})`. The interface also carries a small pubkey↔peerId association table (`associatePeerWithPubkey`, `getPeerIdForPubkey`, `getPubkeyForPeerId`), because a transport addresses peers by an opaque, transport-local peer id (a GATT path id for BLE, an address for UDP), whereas the layers above reason in terms of the peer's public key — its true identity.

`broadcast` is the managed-flooding primitive: it fans a packet out to every currently-connected peer and returns the count sent, taking an optional `excludePeerIds` set so a relay can avoid echoing a packet back down the leg it arrived on. The mesh-routing logic that decides *when* to broadcast (TTL, dedup, store-carry-forward) lives one layer up in the router, not in the transport — the transport only exposes the fan-out mechanism. This split is the same on both transports, which is what lets the router treat them uniformly.

A transport moves through a fixed six-state lifecycle (`TransportState`): `uninitialized` → `initializing` → `ready` → `active`, plus the two out-of-band states `error` and `disposed`. `initialize()` advances a transport from `uninitialized` to `ready` (subscribing to plugin streams and initialising the underlying BLE plugin; permission provisioning happens one layer up, in `GrassrootsNetwork`, not inside the transport); `start()` advances `ready` to `active` (it begins advertising and scanning). The interface defines a single derived predicate, `isUsable`, true exactly when the state is `ready` or `active` — both are states in which the transport can carry traffic, the difference being whether it is also actively seeking new peers. `dispose()` is terminal: a disposed transport is unusable and cannot be revived.

Two properties of this design are load-bearing for the rest of the system. First, the two transports are strictly independent: BLE and UDP are toggled separately and neither's lifecycle state, peer reachability, or online status is allowed to influence the other's. A peer reachable over UDP stays reachable regardless of what BLE is doing, and vice-versa; consolidation into a single "is this peer reachable at all" signal happens above the transport, not inside it. Second, the enum `TransportType` has exactly two values — `ble` and `udp` — and both are real transports; there is no third. An earlier WebRTC transport was removed outright, leaving behind no enum member, no service file, and no stub, in keeping with the project's no-legacy discipline.

1.4.2 BLE Discovery: A Rotating, Pubkey-Derived Service UUID

A device makes itself discoverable by advertising a BLE service UUID whose fixed prefix marks it as a Grassroots device and whose suffix rotates every 15 minutes. `GrassrootsIdentity.deriveServiceUuidForSlot` (`lib/src/models/identity.dart`) concatenates a fixed 8-byte *grassroots prefix* — the first 8 bytes of SHA-256(`"grassroots"`), the constant `84c403160871e5ad` — with the first 8 bytes of `SHA-256("grassroots ble suffix" | pubkey | slot)`, where `slot` is the wall-clock time divided by a 15-minute `bleSlotDuration`, encoded 8-byte big-endian. The derivation throws if the pubkey is shorter than 32 bytes; there is no lenient fallback. The prefix never changes, so a scanner recognises a Grassroots advertisement by prefix-matching; the suffix, however, rolls each slot. This is a deliberate *unlinkability* property: a passive observer who does not know the public key sees a beacon that changes every 15 minutes and cannot track the device across slots, while anyone who *does* know the key — a friend, or the device itself — recomputes the suffix and recognises it. Because unsynchronised clocks and slot boundaries would otherwise break recognition, matching is against a *set*: `candidateServiceUuids` returns the previous, current, and next slot's UUIDs, and `serviceUuidMatchesPubkey` is the recognition primitive the transport uses throughout. The GATT characteristic UUID (`0000ff01-...`) is the same constant across all peers; the peripheral hosts its data service under whatever slot UUID it currently advertises, so when the slot advances a 30-second timer re-advertises the new beacon — which rebuilds the peripheral GATT service under the new UUID, dropping and re-establishing its live central links across the boundary. That rebuild is the accepted cost of an unlinkable, rotating beacon.

The UUID is a discovery hint, never an authorization proof — it tells a scanner "a Grassroots device carrying *this* key is nearby," but authentication is established separately by the self-signed, neighbour-local ANNOUNCE (covered in *Identity, Trust & the Security Model*). Scanning uses the shared grassroots prefix as a service-UUID filter with a continuous window (timeout `Duration.zero`) and `allowDuplicates: true`, so iOS keeps delivering RSSI and liveness updates rather than reporting each device only once. Because a silently-muted scan is a known BLE failure mode, a watchdog fires every 10 s and restarts a scan that has heard nothing for the `scanSilenceRestart` window (default 30 s). Advertising is started with the identity's derived service UUID, the fixed characteristic UUID, and `bondless: true`.

1.4.3 Dual-Role Convergence Over GATT

A single GATT link is one-directional at the role level — one side is central, the other peripheral — so a Grassroots pair must converge to a *dual-role* connection: two legs, each device central on one and peripheral on the other. This is mandatory; a single-link pair is only ever tolerated as a transient state that the transport keeps trying to upgrade. Path ids are role-prefixed accordingly: `central:<remoteId>` for outbound legs we dial, `peripheral:<remoteId>` for inbound legs we accept; only `central:` paths are ever dialed.

Who dials first is decided by advertisement-driven arbitration rather than by leaving a

leg unopened. For two same-platform peers, the tiebreaker is deterministic: the device whose current-slot service UUID sorts lower (`compareTo < 0`) opens the first central leg, mirroring UDP's "smaller pubkey initiates." (Both peers compare the same pair of advertised UUIDs once they are on the same 15-minute slot; a rotation-boundary disagreement simply falls through to the first-mover fallback below.) Platform asymmetry is handled by a marker rather than an exception: iOS devices advertise the fixed local name `grs-ios`, and a peer that sees it yields the first dial to the iOS side. This routes around the one *measured* hardware constraint — an iOS central cannot open the *second* link toward a non-iOS peer (the connect wedges until timeout) — by making iOS always own a pair's first leg and the non-iOS side open the reverse leg. A cold-start "first-mover fallback" (default 5 s) ensures the non-initiator dials anyway if the expected initiator never moves.

Two defensive mechanisms guard the connection machinery against the noise of BLE radio-address privacy. The OS rotates a device's radio MAC / resolvable private address roughly every 30 s, which produces a fresh transport-level path id for the *same* logical peer; the transport suppresses these duplicates, dropping an advertisement from a rotated MAC when a live or in-flight path to the same peer already exists — matched against the peer's candidate slot UUIDs (`candidateServiceUuids`: the current plus adjacent slots), since the service UUID itself now rotates every 15 minutes. Separately, the number of simultaneous in-flight central dials is capped at 2, so a chatty MAC-rotator cannot exhaust the local connection slots. In closed cold-call mode the transport additionally refuses to dial or connect to any peer whose advertised service UUID does not match an accepted friend's candidate slot UUIDs.

1.4.4 Moving Packets, MTU, and the Foreground Service

Once legs are up, sending is direct GATT writes. `sendToPeer` writes to a single ready path and returns `false` if none exists; `broadcast` writes to every ready path, ordered by RSSI descending (peripheral-role paths with unknown RSSI sort last via a -100 fallback but still receive), and returns the count sent. A path is eligible only when its plugin state is `ready` and `path.canSend` is true. Inbound bytes are parsed by `GrassrootsPacket.deserialize`; a malformed packet is caught and silently dropped rather than propagated. On every central connect the transport requests an ATT MTU of 247 bytes — the largest most Android stacks negotiate — because a single ANNOUNCE alone is $\sim$200 bytes, far over the default 23-byte ATT MTU (a 20-byte payload); the effective MTU is whatever the peer accepts, reported back via `BlePath.mtu`.

Because Android aggressively freezes backgrounded processes, the transport pins itself alive with a native foreground service, controlled over the `grassroots/foreground_service` Method-Channel (`start`/`stop`) and declared with the `connectedDevice` foreground-service type. It is a no-op on every platform except Android and swallows a missing native handler, so the same Dart code runs unchanged on iOS.

1.4.5 A Fossil Worth Flagging

One stale comment deserves an explicit warning to any reader who opens the source. The `BleTransportService` class doc still reads "*Direct delivery only. No relaying, no store-and-forward*" — a fossil from before the mesh migration that directly contradicts the project's opportunistic-mesh design. It is technically accurate about *this file*: the BLE transport genuinely contains no relay, flooding, DTN, TTL, or Bloom-filter logic; `broadcast` and `sendToPeer` only push to directly-connected GATT paths. But the comment misleads about the *system*: multi-hop managed flooding and store-carry-forward live one layer up in the router (see *Mesh Routing*), which drives the transport's `broadcast` primitive to achieve reach the transport itself is deliberately ignorant of. The comment should be read as describing the transport's narrow responsibility, not

the mesh's behaviour.

1.5 The Internet Transport & NAT Traversal

Where the BLE mesh (described in *The BLE Mesh Transport* and *Mesh Routing*) gives a device local, Internet-free reach through multi-hop flooding, the Internet transport gives it *global* reach — but only ever as a *direct*, point-to-point path between the two endpoints. There is no store-carry-forward, no relaying, and no caching on this side: a message to an unreachable peer fails immediately rather than being held (`lib/src/transport/udp_transport_service.dart`). Everything that makes the Internet path hard is therefore concentrated in one problem: two mobile devices, each behind a NAT, have to *find* each other and punch a hole through their respective NATs before that direct path can exist. This section covers the transport itself, then the NAT-traversal machinery — hole-punching, signaling, and the facilitators that broker the punch.

1.5.1 The UDX Transport: Sessions, Sockets, and Reframing

The Internet transport is built on UDX, a reliable stream protocol layered over UDP. Its `TransportState` lifecycle follows the two-phase pattern shared with the BLE transport: `initialize()` binds the underlying sockets and moves the transport `uninitialized→initializing→ready`, then `startMultiplexer()` constructs the `UDXMultiplexer` and moves it `ready→active` (`udp_transport_service.dart`). The split matters for hole-punching: raw punch bursts and the multiplexer contend for the same socket, so the transport can hold in `ready` (socket bound, multiplexer not yet running) precisely when it wants to read raw punch datagrams itself.

Initialization binds a `RawDatagramSocket` for *both* address families — IPv6 first, then IPv4 — each on the wildcard address with ephemeral port 0, and each family gets its own socket and its own multiplexer stored in per-family maps. Binding fails only if *neither* family binds. When a single "active" socket must be chosen (e.g. for hole-punch sends), IPv6 is preferred, falling back to the first bound family (`udp_transport_service.dart`).

An outgoing connection (`connectToPeer`) creates a `UDPSocket` on the multiplexer, opens a `UDXStream` with a per-multiplexer monotonically increasing stream id, and awaits `handshakeComplete` under a default 10-second timeout (`_defaultHandshakeTimeout`). The stream id is *never reused* — it increments on every connect — because UDX will refuse a SYN carrying a recently-closed stream id. Failures are classified into `UdpConnectFailureKind.{networkUnreachable, handshakeTimeout, other}`: a `TimeoutException` becomes `handshakeTimeout`, and `SocketException` codes 101/113/65 (or unreachable-route text) become `networkUnreachable`, so the coordinator can distinguish "no route" from "peer silent" (`udp_transport_service.dart`).

Because the wire envelope is *sender-anonymous* (recipient-only, no sender field — see *The Wire Format*), an incoming stream cannot be attributed from its bytes. The transport therefore keys the fresh connection by a temporary key of the form `remoteAddress:remotePort:streamId`, and only once the coordinator verifies the first packet does it call `mapIncomingConnectionToPubkey` to bind that temp key to the peer's public key. Established connections are then keyed by pubkey hex — and since a peer id *is* its pubkey hex, `getPubkeyForPeerId` is just a hex decode (`udp_transport_service.dart`).

UDX delivers a reliable *byte stream*, not discrete datagrams, so the receiver must recover packet boundaries. A `_PacketFramer` buffers incoming bytes and reframes them into 58-byte-header `GrassrootsPackets`: it waits until it has at least a full header, peeks the on-wire payload

length, and slices off exactly `headerSize` + payload bytes before repeating. This is the point where the same 58-byte packet model used on the mesh is reconstructed from a stream. There is also a raw-UDP fallback (`sendRawTo`) that bypasses UDX entirely — used when the UDX handshake fails (e.g. a LAN hairpin) — carrying no reliability, so app-layer ACK retries must cover loss. On the raw receive path, 36-byte `BCPU` punch packets and anything under 50 bytes are dropped as not meaningful (`udp_transport_service.dart`).

1.5.2 Address Candidates and Public-Address Discovery

Consistent with the *one address per connection, multiple candidates per peer* principle, a `_PeerConnection` stores exactly one `addr+port`; `connectToPeer` records a single `'$remoteHost:$port'`→pubkey mapping. There is no per-message address selection and no mid-stream switching. Candidate *selection* is delegated up to the connection logic and expressed as a dial-ability filter: `canDialAddress` accepts an address only if a socket is bound for its family, and either the address is loopback or the transport has a usable route. The set of candidates a peer offers — public IPv4, public IPv6, and link-local IPv6 — arrives via that peer's ANNOUNCE; the transport's job is to filter and dial, not to enumerate (`udp_transport_service.dart`).

Public addresses are discovered by `PublicAddressDiscovery` through an external reflector: `https://ipv6.seeip.org` for IPv6 and `https://ipv4.seeip.org` for IPv4, each behind a 5-second connect timeout, with results cached per family for 5 minutes. A link-local IPv6 candidate (`fe80::`) is additionally harvested from local interfaces and rendered as `'[fe80::...%iface]:port'`; it lets two devices on the same LAN connect directly — bypassing AP client isolation and NAT altogether — and is scoped to the local link, never reaching the public Internet (`public_address_discovery.dart`). The transport also self-heals: `probeAndRebindIfDead` sends a 1-byte loopback probe per family and, on any failing family, tears down and fully re-initializes the whole UDP stack (surfacing per-peer disconnects) — a workaround for Android sockets that die with `EPERM`.

1.5.3 Hole-Punching

A hole-punch packet is a fixed 36 bytes: a 4-byte magic `BCPU` (`0x42 0x43 0x50 0x55`) followed by the sender's 32-byte Ed25519 public key *in cleartext* (`hole_punch_service.dart`). The `HolePunchService` caches one serialized punch packet at construction (identical for every target) and sprays it: `punch()` is fire-and-forget for a default 2 seconds at a 200 ms interval, while `punchUntilResponse()` sends at the same interval and returns on the first inbound punch, or false after a 5-second timeout. Sending is what matters: the act of transmitting opens the sender's NAT mapping, so a receiver need not process the packet at all — and indeed the embedded pubkey is only a NAT-mapping *hint*, not authenticated, since punch packets are unsigned.

It is important to state plainly what this service does *not* do. `HolePunchService` contains no notion of a deterministic initiator, no simultaneous-punch scheduling, no `PUNCH_INITIATE` handling, and no friend-gating — it only sprays bursts. The idealized well-connected-friend flow (both sides punch at a coordinated instant) is *partial* and split across layers: the coordination that decides *when* and *toward whom* to punch lives in the coordinator and the `SignalingService`, described next, not in this service.

1.5.4 Signaling: The Reconnection Protocol

Signaling is the out-of-band control channel that lets two NAT'd peers rendezvous. Its messages are defined by `SignalingType` in `lib/src/signaling/signaling_codec.dart`, an enum whose wire values *start at 0x05*: `punchInitiate(0x05)`, `punchReady(0x06)`, `addrReflect(0x07)`, `reconnect(0x08)`, `available(0x09)`, `rvList(0x0a)`, `friendList(0x0b)`. Values `0x00`–`0x04` are permanent gaps: the

earlier ADDR_QUERY/ADDR_RESPONSE/PUNCH_REQUEST types were deleted outright when the flow switched to RECONNECT/AVAILABLE matching — a clean break that, per the project's no-legacy rule, leaves holes rather than renumbering.

The core reconnection handshake is deliberately two-sided. When peer A's address changes, A sends RECONNECT(target=B); peer B independently sends AVAILABLE(target=A). A facilitator that has observed each sender's live source ip:port matches A's RECONNECT against B's AVAILABLE (the match key is the inverse of the pending key) and, on a match, sends each side a PUNCH_INITIATE carrying the *other* side's observed address. Both peers then punch toward those addresses; PUNCH_READY is exchanged so each learns the hole is open, and ADDR_REFLECT is the STUN-equivalent by which a facilitator reflects a peer's own observed external address back to it. Mediation is family-constrained — the target address is chosen to match the requester's IP family (IPv4 or IPv6), and the punch is aborted if none matches — and duplicate coordinations for the same (sorted) pubkey pair are suppressed within a 15-second cooldown (signaling_service.dart).

Two facts anchor the trust model. First, *signaling is authenticated end-to-end, not per-packet*: a signaling message travels inside a Noise-sealed secure envelope carrying ContentType.signaling (see *The Noise Session Layer*), so the sender is authenticated by trial-decrypt and the anonymous outer envelope carries no signature. As a defence-in-depth check, RECONNECT additionally duplicates the initiator pubkey inside its 64-byte payload and the receiver rejects it unless that inner value byte-matches the authenticated sender. Second, *signaling is friend-only and signaling-only.* Untrusted senders are dropped *before* decode: a sender is trusted only if it is a friend, a configured rendezvous server, or a known friend RV server. And a facilitator only ever moves *metadata* (addresses and punch timing) — never message content. This is the trust boundary that distinguishes the Internet path (friend-gated signaling; content stays direct and end-to-end) from the BLE mesh (open relay of sealed bytes for arbitrary recipients).

1.5.5 Facilitators: Friend Mediators vs. Rendezvous Servers

Two kinds of node can broker a punch, and they run different subsets of the protocol. A *friend mediator* is an ordinary well-connected friend that both peers happen to know; it runs *only* the single-step RECONNECT path, and it drops the target unless that target is a friend *and* currently reachable on the mediator. It explicitly refuses AVAILABLE and the two-sided matcher — "friend mediators do not run the rendezvous-server matcher" (signaling_service.dart). The two-sided RECONNECT↔AVAILABLE matcher belongs to a *rendezvous server.*

The rendezvous server is implemented today as the bootstrap_anchor: a publicly-reachable, well-connected reference node with its own independent Ed25519 keypair, no friends list, and no place in the social graph (bootstrap_anchor/lib/src/anchor_server.dart). It listens on IPv6 and IPv4 (default port 9516), observes each cold-caller's source ip:port, and runs the full matcher; it drops any PUNCH_INITIATE/ADDR_REFLECT/RV_LIST sent *to* it (it is well-connected and knows its own address), and it *never relays messages or fragments* — message and fragment packets are dropped with "rendezvous server does not relay messages". It, too, requires signaling to arrive Noise-sealed: plaintext signaling is refused, and it acts purely as a Noise responder, never initiating a session. Peers learn each other's rendezvous servers by exchanging RV_LIST (a pubkey→address map recorded per friend) and FRIEND_LIST (the friends-of-friends map that lets a well-connected friend qualify as a facilitator for a given target). One honesty caveat in the current anchor: friendship-proof verification is *intentionally deferred* — it matches any signed RECONNECT/AVAILABLE pair, so the "verifies friendship proofs" promise in its own doc comment is not yet backed by code.

1.5.6 Address Persistence and Future Work

A crosscutting invariant governs stored addresses: a peer's last-known address is *never unilaterally cleared*. It is updated when a newer valid address arrives (from ANNOUNCE, signaling, or direct observation) and cleared only when the peer itself signals it no longer has one. Stale-peer cleanup, our-side disconnects, and transport rebinds must not null it out, because it is the only handle for attempting reconnection. The client-side address table is in-memory and volatile — lost on restart, with friends re-registering on startup — and supports multiple candidates per peer, returned newest-first (`address_table.dart`). The anchor's table mirrors the same discipline, protecting live-connected peers from stale eviction and treating the live source address as the source of truth over stale ANNOUNCE state.

Finally, the boundary between shipped and planned must be drawn precisely here. *Rendezvous servers are the current, live NAT-bootstrap mechanism*: the `bootstrap_anchor`, `RendezvousServerSettings`, `RV_LIST`, and the configured-RV plumbing are all fully implemented and in use. The design direction of *replacing* rendezvous servers with well-connected friends — along with signed `grassroots://` invite links to bootstrap first contact — is recorded as a design note but is *not* implemented: the invite-link surface (URL schemes, deep-link handlers) is absent from the build configuration. Two further items are likewise future work: the from-scratch multi-hop Noise handshake (the IK/KK patterns) is not built — handshakes are neighbor-local today — and full simultaneous-punch coordination remains partial across the coordinator and `SignalingService` rather than a single orchestrated flow.

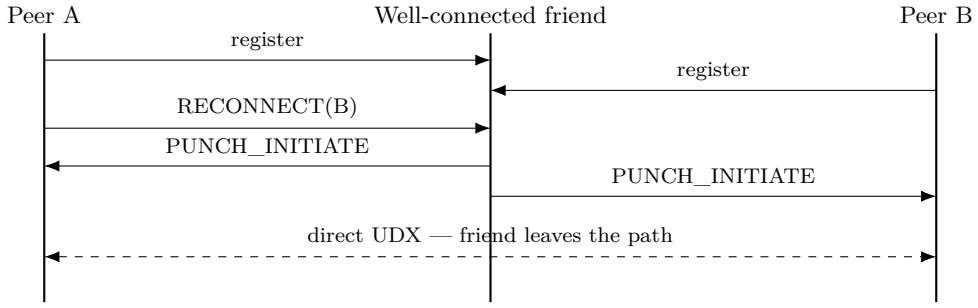


Figure 1.2: NAT hole-punch mediated by a well-connected friend. The friend relays only addresses and `PUNCH_INITIATE`; the deterministic initiator (lexicographically smaller pubkey) then dials the direct UDX stream.

1.6 Mesh Routing: Managed Flooding & DTN Store-Carry-Forward

The Bluetooth mesh is where Grassroots earns the word *opportunistic*: a message reaches a friend who is not a direct neighbour by hopping across intermediaries, and a message for a friend who is momentarily out of range is carried until they reappear. Both behaviours live in one place. Every inbound packet, from every transport, enters the router through the single method `processPacket` (`lib/src/routing/message_router.dart:148`). This is deliberate: BLE-arrived and UDP-arrived packets are demultiplexed by the same dispatch, deduplicated against the same filter, and (for BLE) relayed by the same rule, so there is exactly one definition of "what happens to a packet." As established in *Wire Format & Packets*, the outer header the router sees is sender-anonymous — recipient ID, type, TTL, packet ID, and length only — so every decision below is made without knowing who originated the packet, and without being able to read its contents.

The dispatch: announce and handshake are special

`processPacket` first peels off the two packet types that are *not* mesh traffic. An `announce` is handed to the ANNOUNCE handler and returned early — never deduplicated, never relayed (`message_router.dart:160`). A `noiseHandshake` is checked as addressed-to-us and dispatched, then likewise returned early (`message_router.dart:175`). Both are minted with TTL 1 at the coordinator (`grassroots_network.dart:4930` for ANNOUNCE, `2789` for the handshake), which makes them structurally **neighbour-local**: a TTL-1 packet can never satisfy the relay condition (which requires `ttl > 1`), so it dies at the first hop by construction. The rationale is that both are about *presence and pairwise key agreement*, not reach: ANNOUNCE advertises "I am here, and here is my signed identity" to immediate neighbours, and the Noise `XX` handshake — as detailed in *The Noise Session Layer* — is today a neighbour-local exchange. A from-scratch multi-hop handshake (Noise `IK`/`KK` carried across relays) is **future work**; the current handshake reaches only direct neighbours, which is why an existing session, not a fresh handshake, is what lets the flood carry a message to a non-adjacent recipient.

Managed flooding

Everything that survives the early returns — i.e. every `secure` envelope — is a mesh packet. The router deduplicates it by calling `checkAndAdd` on the outer `packet.packetId` against a `BloomFilter` of recently-seen IDs; `firstSeen` is the negation of "was already present" (`message_router.dart:189`). A packet is relayed only when three conditions hold simultaneously (`message_router.dart:208`):

1. **First sight.** `firstSeen` is true — a duplicate is dropped, killing loops and re-floods.
2. **TTL budget.** `packet.ttl > 1`. The relay forwards `packet.decrementTtl()`, so the TTL falls by one per hop and the packet is dropped at the last hop (TTL reaching 1). The default starting TTL is 7 (`packet.dart:60`), bounding any packet to at most seven hops of spread.
3. **Rate budget.** The per-neighbour relay budget (below) permits it.

Relaying is **open**: it is not friend-gated. The router forwards a sealed packet for a recipient it has no relationship with, because it can only see the recipient ID and opaque ciphertext — it cannot read the body, and the header carries no sender to authorise against. This is the deliberate reversal that gives the mesh its reach, and it is safe precisely because a relay handles nothing but recipient-addressed sealed bytes.

Crucially, a relay forwards the packet's bytes *unchanged*. The router hands the decremented packet to the coordinator's `onRelay` callback (`grassroots_network.dart:3894`), which re-floods over BLE only — UDP stays direct point-to-point, per *The Internet Transport* — via `_floodViaBle(packet.serialize())` and thence `BleTransportService.broadcast(bytes, excludePeerIds)`, writing that one identical, already-sealed ciphertext to every ready BLE neighbour. A relay *cannot* re-seal, because it cannot decrypt; it copies opaque bytes. This is distinct from the coordinator's own `broadcast(payload)` method, which does seal a *fresh* copy per neighbour under each peer's Noise session — but that path sends plaintext the coordinator itself holds, and is not relaying. On the relay path the inbound BLE peer is excluded from the fan-out (`excludeBlePeerId`, `message_router.dart:210`) so a packet is never echoed straight back to the neighbour it arrived from.

The Bloom filter and its hard-clear rotation

The dedup filter (`lib/src/mesh/bloom_filter.dart`) is 96,000 bits with 7 hash functions (double-hashed: FNV-1a plus a Murmur3-like second hash). It self-rotates by a **hard full clear** — not a generational scheme — when either the item count reaches 10,000 or 5 minutes elapse since the last rotation (`bloom_filter.dart:110`). The consequence worth naming honestly: a packet ID seen just before a rotation can be treated as new just after it, so the same packet could in principle be re-processed or re-relayed across a rotation boundary. This is an accepted trade-off — the filter must be bounded in both memory and staleness, and TTL plus the rate budget cap the damage of any re-flood. ANNOUNCE and handshakes never touch this filter at all, having returned early.

Bounding the flood: the per-neighbour relay budget

An open flood without a rate cap is a battery sink and an abuse vector. Each attributable inbound neighbour is limited to 512 relays per 10-second window (`message_router.dart:43`); once the window's count is exhausted, further relays from that neighbour are refused until the window rolls over. The important caveat is attribution: `_allowRelayFrom` returns `true` unconditionally when the inbound peer id is `null` — the unattributable case, e.g. UDP (`message_router.dart:521`). So the 512/10 s cap governs *BLE neighbours*, which is where flooding actually happens; UDP relays are not per-neighbour rate-limited (and in practice UDP is direct point-to-point, so this branch rarely relays at all).

DTN store-carry-forward

Flooding only helps if a path to the recipient currently exists. When it does not, the router falls back to delay-tolerant carriage. After — and only after — a successful relay decision, if the recipient is not currently reachable, the sealed packet is cached in the DTN store keyed by recipient hex (`message_router.dart:217`). This ordering matters: if the per-neighbour budget blocked the relay, the packet is *not* DTN-cached either, so a rate-limited node does not silently become an unbounded buffer. The store (`lib/src/mesh/dtn_store.dart`) is bounded on three axes, each an explicit anti-abuse bound:

- **Age.** Entries older than 6 hours are pruned on every `store` and `takeFor`.
- **Per-recipient depth.** At most 32 packets per recipient; the oldest are evicted first. Storage is idempotent per packet ID, so re-seeing the same packet does not grow the bucket.
- **Recipient count.** At most 256 distinct recipients; when exceeded, the store evicts the recipient whose oldest packet is oldest.

Re-delivery is event-driven. When a recipient reappears (on their ANNOUNCE / peer-connected event), the coordinator calls `flushDtnFor`, which takes all non-expired cached packets for that recipient and re-floods each via `onRelay` (`message_router.dart:630`) — this time with no excluded peer, since it is a fresh flood rather than a forward of one specific inbound copy.

Two distinct dedup keys

Because a large message is fragmented, the router keeps two separate dedup domains. Relay and loop suppression run on the **outer** `packetId` (pre-decrypt), so an intermediary can dedup without reading anything. Message *delivery* dedup runs on the **inner** `messageId` recovered from

the reassembled `SecureFrame` after decryption (`message_router.dart`, reassembly path). Thus a first-seen fragment is relayed, but the application receives the assembled message exactly once; a duplicate that still arrives is re-ACKed without being re-delivered.

The sender's own queue is not the relay's DTN

It is important not to conflate two carry mechanisms that look superficially alike. The *relay* DTN store above holds *other peers'* sealed traffic opaquely. The *sender*, by contrast, keeps its own outbound responsibility: every successful `send()` registers the message in an ACK-pending table with a watchdog `Timer` of `ackTimeout` (default 5 s, `grassroots_network.dart:2277`). On timeout — or immediately when the BLE path it was riding drops — the message is re-queued onto the sender's per-recipient outbound FIFO (`grassroots_network.dart:2290`, 2327), and the periodic announce timer drains that queue for peers that have become live. This is the madGLP "fair message delivery" assumption realised at the origin: the sender re-floods its own un-acked messages when conditions change, independent of whether any relay chose to cache them. The relay's DTN cache is a best-effort community amplifier; the sender's queue plus ACK watchdog is the actual delivery guarantee. Bounding every one of these structures — TTL, Bloom size and rotation, the relay budget, and all three DTN caps — is the same principle applied throughout: an unbounded flood or cache is an abuse and a battery drain, so nothing in the mesh is allowed to grow without a ceiling.

1.7 The Noise Session Layer

Every end-to-end guarantee in Grassroots rests on a single component: a hand-rolled implementation of the Noise Protocol Framework's `XX` handshake, living in `lib/src/session/noise_session_manager.dart`. The transport deliberately owns this code rather than deferring to a black-box library, because the mesh imposes requirements a generic Noise library does not anticipate: sessions must be demultiplexed without a sender field on the wire, they must survive a peer moving between relay paths and transports, and the transport-message AEAD must bind exactly those envelope fields a relay may *not* mutate. This section explains how the layer meets those requirements, and marks clearly where today's implementation is neighbour-local and a broader design remains future work.

Pattern, roles, and the three-message handshake

The concrete protocol is `Noise_XX_25519_ChaChaPoly_SHA256` (`noise_session_manager.dart:11`): the `XX` handshake pattern, X25519 Diffie–Hellman, ChaCha20-Poly1305 for authenticated encryption, and SHA-256 for hashing. `XX` is a mutual-authentication pattern in which neither party knows the other's static key in advance — both static keys are transmitted, encrypted, *during* the handshake. This matches the mesh's trust model, where a node may cold-call a neighbour it has never spoken to.

The handshake is exactly three messages, enumerated `message1(1)`, `message2(2)`, `message3(3)` (`noise_session_manager.dart:18-21`), exchanged between an `initiator` and a `responder` (`NoiseHandshakeRole`, line 16). The initiator writes messages 1 and 3; the responder writes message 2. Each message has a fixed, strictly-checked size, so a malformed handshake fails fast rather than degrading:

- **Message 1** — 32 bytes: the initiator's ephemeral X25519 public key, sent in the clear (`noise_session_manager.dart:417-419`).

- **Message 2** — 80 bytes: the responder's ephemeral (32 bytes) plus its encrypted static key (32-byte key + 16-byte MAC = 48 bytes) (`noise_session_manager.dart:442-445`).
- **Message 3** — 48 bytes: the initiator's encrypted static key (32 + 16) (`noise_session_manager.dart:473-475`).

Each handshake message is wrapped in a small versioned frame, `[version=1][message-value][body]` (`noise_session_manager.dart:710-732`), and a decode of fewer than two bytes or a wrong version byte is rejected — there is no tolerance for a truncated or older frame, in keeping with the project's no-compatibility-shim rule.

Symmetric-state primitives

The manager implements the Noise symmetric-state machine directly (`noise_session_manager.dart:534-612`). `initialize` seeds a 32-byte hash from the protocol name; `mixHash` folds new data into that running transcript hash as SHA-256(hash ‖ data); and `mixKey` runs a two-output HKDF-SHA256 (`_hkdf2`, lines 688–708) to derive a fresh chaining key and cipher key, resetting the per-key nonce to zero. `encryptAndHash` / `decryptAndHash` apply ChaCha20-Poly1305 using the current transcript hash as AAD (lines 572–577), so every ciphertext is bound to everything exchanged so far. A subtle but correct detail is the **pre-first-`mixKey` passthrough branch**: while no cipher key exists yet (`cipherKey == null`), `encryptAndHash`/`decryptAndHash` pass their data through as cleartext and merely `mixHash` it (lines 565–571, 584–588). This is exactly why message 1 (the initiator ephemeral) travels unencrypted — there is no key material to protect it yet under `XX` — and it is authenticated retroactively by the transcript. Finally, `split` derives the two directional transport keys from an HKDF-2 over the final chaining key, with the initiator's send/receive keys being the responder's receive/send keys (lines 605–611), giving each direction its own key.

Identity inside the handshake, and the birational check

Grassroots identity is an Ed25519 key pair, but Noise operates over X25519. The manager bridges the two with libsodium's birational map: the local Noise static is derived from the Ed25519 identity via `skToCurve25519`/`pkToCurve25519` (`noise_session_manager.dart:307-316`). Crucially, the peer's identity is never carried in a cleartext header — it arrives *inside* the encrypted static transmitted during the handshake. To prevent a peer from presenting a Noise static that does not correspond to its claimed Ed25519 identity, `_verifyRemoteStatic` recomputes `pkToCurve25519(remotePubkey)` and compares it, in constant time (`_bytesEqual`, lines 347–354), against the static Noise actually delivered (lines 331–345). If they disagree — or the point is invalid — verification fails and no session is formed. This is the load-bearing authentication check: there is **no per-packet Ed25519 signature** anywhere in this layer, correcting a stale class-doc remark that lists "signing packets" as a use of the identity (`identity.dart:14-15`); only ANNOUNCE is payload-signed, and the identity's private key is used here purely for this static-key derivation.

Keying by peer, and sender-anonymous demultiplexing

Sessions are stored in a map keyed by the peer's Ed25519 public key rendered as hex (`noise_session_manager.dart:64-67`, `_hex` at 804–806) — *not* by transport path or socket. A session therefore transparently survives a peer switching relay paths on the BLE mesh or moving between BLE and Internet transports; the same key material serves every path. Because the outer envelope is sender-anonymous (recipient-only, no sender field), an inbound `secure` packet carries nothing

that says which session decrypts it. The manager resolves this by **trial-decrypt** (`trialDecrypt`, lines 170–193): it iterates every active session, attempting an AEAD open; the Poly1305 tag succeeds for at most one session, which both selects the session and recovers the sender's identity as a by-product. The call returns the cleartext packet together with the recovered sender pubkey, or `null` if no session matches. On the send side, `encryptPacket` seals only `secure`-typed packets and leaves the type as `secure` (the post-refactor design no longer swaps in "secure-variant" packet types), so relays observe an identical opaque type for every content class.

Transport AEAD, AAD, and replay

An encrypted transport payload is laid out as `[version=1][8-byte little-endian nonce][ciphertext ‖ 16-byte MAC]` (`noise_session_manager.dart:627-644`), the 64-bit counter being written little-endian at offset 4 of a 12-byte ChaCha20-Poly1305 nonce buffer (`_aeadNonce`, lines 738–743). The associated data (AAD) is 81 bytes: the constant `secure` wire type (1) + sender pubkey (32) + recipient (32) + packetId (16) (`_applicationAad`, line 766). It **deliberately excludes `ttl` and `timestamp`**, precisely because a relay decrements TTL in flight — binding TTL would break end-to-end authentication the moment the packet was forwarded. The content type is *not* in the AAD because it no longer lives on the wire at all; it sits inside the AEAD-protected plaintext as a `SecureFrame`, and is thus authenticated by the ciphertext itself (see *The Wire Protocol*). Replay is resisted per direction by a bounded set of received nonces capped at 2048 with FIFO eviction (lines 661–663, 679–685); a duplicate nonce raises a `StateError`. This is a *bounded* guard, not an absolute one — a nonce evicted after 2048 newer arrivals could in principle be replayed — a documented trade-off between memory and strictness.

Establishment scope, timeouts, and future work

Handshake liveness is enforced by a 5-second timeout on `waitForSession`: on expiry the half-built session entry is reset and `false` returned (`noise_session_manager.dart:74, 112-118`). Simultaneous initiation — both peers sending message 1 at once — is broken deterministically by comparing local and remote pubkey hex: the lexicographically-smaller side keeps its initiator handshake and ignores the inbound message 1, the other abandons and becomes responder (lines 213–219).

Today, handshake establishment is strictly **neighbour-local**: both ANNOUNCE and `noiseHandshake` packets are sent with `ttl: 1` and are never flooded, and `remotePubkey` is resolved by the coordinator from the neighbour's verified self-signed ANNOUNCE on the inbound BLE path rather than from the packet (which carries no sender) (`noise_session_manager.dart:124-127`). A session, once formed, is end-to-end and survives relaying — but forming one requires the two endpoints to be direct neighbours. A from-scratch multi-hop handshake using the `IK`/`KK` Noise patterns, which would let two non-adjacent mesh nodes establish a session across relays, is **future work** and is not implemented.

1.8 State Management: the Redux Projection

All shared state in the transport lives in a single immutable Redux store. The root of that store is `AppState`, an immutable value composed of exactly six slices (`lib/src/store/app_state.dart`): `transports`, `peers`, `messages`, `friendships`, `settings`, and `signaling`. The user interface reads from these slices and rebuilds when they change; the transport layers never mutate application state directly. Instead, every observable event flows through the Redux cycle: a transport observation is turned into an *action*, a pure *reducer* folds that action into a new immutable state, and *subscribers* (the UI, the persistence service) react to the resulting change. This chapter's

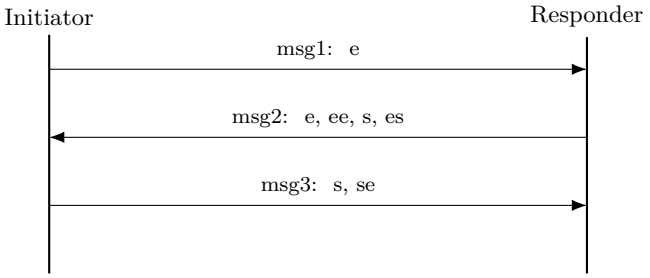


Figure 1.3: The `Noise_XX_25519_ChaChaPoly_SHA256` handshake. Static keys are transmitted encrypted; each side's Ed25519 identity is cross-checked against its Noise static via the birational map. Establishment is neighbour-local (handshake packets carry TTL 1).

discussion of *the coordinator* and *the Noise session layer* describes the producers of these actions; here we describe where their facts land and how they are shaped.

1.8.1 The six slices

Each slice owns one concern and nothing more.

- **PeersState** (`lib/src/store/peers_state.dart`) holds `discoveredBlePeers` (raw BLE discoveries keyed by radio path id), `peers` (identified peers keyed by public-key hex), and a `friendsOfFriends` adjacency map. A `PeerState` records both BLE legs of the mandatory dual-role pair as two separate path-id fields (`bleCentralDeviceId`, `blePeripheralDeviceId`), the peer's UDP address plus a set of advertised `udpAddressCandidates` and a link-local fallback, and derived predicates such as `isReachable`, `hasBleConnection`, `activeTransport`, and `isWellConnected`.
- **MessagesState** (`lib/src/store/messages_state.dart`) holds delivery-status records (`outgoingMessages`/`incomingMessages` keyed by message id), per-conversation chat history (`conversations` keyed by peer hex, oldest first), and `unreadCounts`. It is bounded: `maxMessages`= 1000 and `maxMessagesPerConversation`= 500. Notably this slice carries application-level chat and even view-once picture media as first-class content, which sits a little above the "pure packet transport" framing of the rest of the system.
- **FriendshipsState** (`lib/src/store/friendships_state.dart`) is a map keyed by `peerPubkeyHex`; each entry carries status (`pending`, `received`, `accepted`, `declined`), the friend's last-known UDP address, and a `knownRvServers` map. Receiving a friend request while our own request to the same peer is outstanding auto-accepts, giving a mutual-request handshake.
- **TransportsState** (`lib/src/store/transports_state.dart`) tracks the two transports' lifecycles *independently* through separate `bleState` and `udpState` fields (plus per-transport error strings), our discovered `publicAddress`, and the current `networkConnectionType`. "Well-connected" is defined here: `isWellConnected` is true iff we hold a globally routable public address.
- **SettingsState** (`lib/src/store/settings_state.dart`) holds the user-toggleable configuration: `bluetoothEnabled` and `udpEnabled` (both default true), `transportPriority` (default [bluetooth, udp]), `coldCallTrustLevel` (default `closed`), the debug `bleRoleMode`, trace-logging consent, and the configured rendezvous servers.
- **SignalingState** (`lib/src/store/signaling_state.dart`) is deliberately minimal: a single `holePunchAttempts` map from target peer hex to a `HolePunchStatus` (`requested`, `punching`,

succeeded, failed). Hole-punch coordinates (ip, port, failure reason) ride *on the actions* but are not retained in state — only the status transition is recorded.

1.8.2 The actions → reducers → subscribers cycle

appReducer (lib/src/store/reducers.dart) is the root reducer. It dispatches by the action's base type — PeerAction, MessageAction, FriendshipAction, SettingsAction, TransportAction, SignalingAction — to the matching slice reducer, and returns the state unchanged for any action it does not recognise. Each slice reducer is a pure function of (old slice, action) → new slice, and AppState.copyWith threads the one changed slice into a fresh root. Because the state is immutable, subscribers detect change by value equality; a slice that a reducer leaves untouched is returned identically and triggers no rebuild.

1.8.3 The strict-projection principle

The governing rule is that the Redux state is a *strict projection* of facts the transport layers explicitly emit — never an inference. Reducers must not synthesise state from "I have not heard from peer X in N seconds" heuristics; surfacing a path failure is the transport's job. Two examples make this concrete. StalePeersRemovedAction only *removes* non-friend peers past a staleness threshold and explicitly refuses to touch connectionState, because that field is driven solely by BLE plugin events and UDX events (lib/src/store/peers_reducer.dart); friends are never removed at all. And a UDP disconnect drops reachability *immediately* on the explicit disconnect event rather than by any timeout (lib/src/grassroots_network.dart). State is a record of what the transports reported, not a guess about what they might be doing.

1.8.4 Transport independence and consolidated reachability

The two transports are independent: losing or disabling one must have zero effect on the other. This invariant is encoded directly in the reducers. PeerBleDisconnectedAction preserves hasLiveUdpConnection, and PeerUdpDisconnectedAction preserves both bleAuthenticated and the stored udpAddress (kept for later reconnection) (lib/src/store/peers_reducer.dart). A peer's connectionState flips to disconnected on a transport drop only when the *other* transport link is already absent. Reachability itself is authenticated-only: isReachable is bleAuthenticated $\vee$ hasLiveUdpConnection (lib/src/store/peers_state.dart), so a raw BLE or UDX link without a completed Noise session does not count as reachable. The one deliberate exception is a rendezvous server, whose raw UDX connection is treated as reachability because it runs no Noise handshake.

Application-level connectivity is reported as *consolidated* end-to-end reachability rather than per-transport events. The coordinator's reachability diff (processReachabilityTransitions, lib/src/grassroots_network.dart) compares each peer's current isReachable against the previous tick and fires onPeerConnected/onPeerDisconnected only on a transition. Because reachability is the OR of the two transports, one of two live transports flipping leaves the OR unchanged and fires nothing — a disconnect fires only when the *last* transport drops (a $0 \leftrightarrow N$ live-transport transition). A peer removed from the map while still reachable is surfaced as a synthesised disconnect carrying only the reconstructed identity.

1.8.5 The TransportState lifecycle

Each transport moves through a six-state lifecycle enum (lib/src/transport/transport_service.dart): uninitialized → initializing → ready → active, plus error and disposed.

A transport is "usable" when it is `ready` or `active`. `TransportsState` carries one such enum per transport (defaulting to `uninitialized`) so BLE and UDP progress through their lifecycles wholly independently.

1.8.6 Selective persistence

Persistence is deliberately partial. `PersistenceService` (`lib/src/store/persistence_service.dart`) writes exactly three domains to `SharedPreferences`: friendships, settings, and conversations-plus-unread-counts (under `v2`-versioned keys), debounced by a 500 ms timer with a `flush()` for app exit. The `peers`, `transports`, and `signaling` slices are *intentionally not persisted*. This is the strict-projection principle applied to durability: peer, transport, and hole-punch state are live facts that must be re-derived from the transports at runtime, so restoring a stale snapshot of them would violate the projection. Friendships, user settings, and chat history, by contrast, are durable user data with no live source to re-derive from.

1.8.7 The coordinator as the bridge

`GrassrootsNetwork` (`lib/src/grassroots_network.dart`) is the bridge between the transports and the store. It subscribes to raw BLE and UDX events and to the Noise session layer, and translates them into actions. A completed Noise handshake becomes a `PeerBleAuthenticatedAction` or a `PeerUdpConnectionChangedAction(connected: true)`; a torn-down UDX session becomes the corresponding `connected: false`; an ANNOUNCE becomes address and peer-list updates. This is precisely where explicit transport facts enter the projection, keeping reducers free of any inference.

1.8.8 Honest warts

A few rough edges are worth recording. `lib/src/store/store.dart` is only a library barrel of export directives, not the actual store construction — the store and middleware are wired elsewhere. `AppAction` (`lib/src/store/actions.dart`) is a dead abstract base class: none of the per-slice action base classes extend it, and `appReducer` takes a `dynamic action`, so it is orphaned. Several `hashCode` implementations are intentionally weak: `MessagesState`, `FriendshipsState`, and `SignalingState` all hash only the *length* of their maps, so two structurally different states of equal size collide (equality via `==` remains content-exact). `PeerState`'s equality deliberately excludes `bleAuthenticated`, which means a change to that flag alone is invisible to Redux equality even though `isReachable` depends on it — a latent missed-rebuild risk for reachability-driven UI. Finally, the debug log is a scoped exception to the "no mutable singletons" rule: `DebugLogScreen` and its `LogBuffer` bypass Redux entirely and keep their ring buffer in a mutable global singleton, unlike all other UI state, which reads `AppState`.

1.9 The Diagnostic Trace-Logging Subsystem

The transport described in the preceding sections is intended for deployment on real phones carried by real people, and a central research question behind Grassroots Networking is empirical: how does an opportunistic BLE mesh actually behave in the wild — how far do messages travel, how often are they relayed rather than delivered directly, how dense are the neighbourhoods a device wanders through, and what does all of this cost in battery? Answering these questions demands telemetry gathered from live deployments, not from a simulator. The *diagnostic trace-logging subsystem* exists to collect that telemetry. It is, by deliberate design, a consent-gated, cleanly separable add-on: it is injected into the network core rather than baked into it, it is

a no-op until the user opts in, and its privacy model is engineered so that the research server cannot re-link a device across upload sessions. This section describes what it records, how it protects the participant, and — honestly — where its documentation has drifted from its code.

1.9.1 Separability and consent gating

The subsystem is two small classes, `TraceLogger` (`lib/src/trace/trace_logger.dart`) and `LocationSampler` (`lib/src/trace/location_sampler.dart`), plus a standalone `trace_server`. `TraceLogger` is constructed once in `lib/main.dart` with a platform label derived from `defaultTargetPlatform` (`'ios'` or `'android'`) and is then *injected* into `GrassrootsNetwork` through its constructor (`GrassrootsNetwork(..., trace: traceLogger)`, `lib/main.dart:737-742`). The network core calls `log()` at instrumented points but has no other coupling to the subsystem; removing it would require only deleting those call sites. As the *State Management* section describes, consent itself is Redux state: `TraceLogger.setEnabled` is seeded from the persisted `settings.traceLoggingConsent` flag and kept in sync by a store subscription (`lib/main.dart:296-299`). Until the user accepts a one-time consent prompt — which fires only when `settings.consentTimestamp` is null (`lib/main.dart:592-625`) — the logger is disabled, and `log()` is a guarded no-op that appends nothing and never throws (`lib/src/trace/trace_logger.dart:56-68`). Consent withdrawal calls `clear()`, deleting the live buffer, any pending batch, and its sidecar (`lib/src/trace/trace_logger.dart:146-150`).

1.9.2 The privacy model: rotating, unlinkable identities

The subsystem's core privacy guarantee is that the server cannot correlate a participant across uploads. This is achieved by generating fresh identifiers *per upload* rather than assigning a stable device identity. On each upload, `deviceId` is a brand-new UUID v4 (`trace_logger.dart:95-96`, commented `// rotating per-upload`), and every distinct non-empty peer public key referenced in the batch is replaced by a per-upload alias `p0`, `p1`, ... assigned in first-seen order (`trace_logger.dart:159-169`). Two uploads from the same phone therefore share no `deviceId` and no peer identifier, so the raw pubkeys never leave the device and cross-upload re-linking is defeated at the source. The consequence — accepted deliberately — is that any longitudinal analysis (a device's mesh behaviour over weeks) must be derived *on-device* before upload; the server sees only unlinkable per-batch snapshots.

1.9.3 Record types and fields

Records are free-form JSON maps, each carrying a `type` and, where applicable, an event time `t` in epoch milliseconds. Six types are documented in `trace_server/schema.md`:

- **message** — the richest type, with direction variants `sent`, `recv`, `delivered`, `ack_timeout`, and `dup`. Crucially, after the mesh migration these fields are *real* measurements, not constants: a `recv` carries `relayHops` (computed as `defaultTtl` minus the TTL at receipt, i.e. relays actually traversed) and `deliveryMethod` (`'direct'` for zero relays vs. `'relayed'` for a mesh flood); a `delivered` carries `e2eLatencyMs` and `deliverySuccess` (`schema.md:63-90`). This is exactly the mesh-reach telemetry the research programme wants.
- **contact** — a peer encounter, logged on disconnect from a start timestamp captured on `onPeerConnected` (`lib/main.dart:744-756`).
- **density** — a foreground periodic sample of the local neighbourhood: `peersConnectedNow`, friend count, mean RSSI, and a coarse geo fix (`lib/main.dart:516-534`).

- **visit** — a completed dwell at a geohash cell (see below).
- **buffer** — absolute occupancy of the two bounded queues described in the *Mesh Routing* section: `outboundQueued` (the sender's own outbound queue) and `dtnBuffered` (sealed packets held in the store-carry-forward cache), taken from `GrassrootsNetwork.outboundQueuedCount` / `dtnBufferedCount` (`schema.md:177-184`).
- **device** — battery and lifecycle health, with two emit points: a foreground periodic sampler (`batteryPct`, `batteryDrainPctPerHr`, `lifecycleState`, `networkType`) and an on-resume sample (`bgDurationMs`, `osThrottled`, the latter an approximation defined as `bgDurationMs > 60s`) (`schema.md:154-171`).

Both periodic record families are emitted by a single 60-second timer (`_traceSampleTimer`, `lib/main.dart:425-426`) that runs *foreground-only* — it does no sampling work while the app is backgrounded, so background behaviour is observed only indirectly through the resume-time `bgDurationMs`/`osThrottled` proxy.

1.9.4 Coarse geolocation and geohash visits

Location is deliberately coarsened before it is ever recorded. `LocationSampler.sample()` requests a fix at `LocationAccuracy.low` with a 15-second time limit and returns null on any error or missing permission (`location_sampler.dart:48-60`). The returned fix rounds latitude and longitude to three decimal places and computes a geohash cell at precision 6 (roughly a 1.2 km × 0.6 km cell) using the standard base-32 alphabet (`location_sampler.dart:66-72,105-141`). Permission is requested lazily, once, and only when trace logging is enabled, accepting either `always` or `whileInUse` (`location_sampler.dart:24-35`; `lib/main.dart:471-475`). A `visit` record is emitted only on cell *change* — on leaving the previous cell — and carries `placeId` (the departed cell), `arrivedAt`, `leftAt`, `visitMs`, an optional `returnTimeMs`, and `visitCount` (`location_sampler.dart:79-95`).

1.9.5 Durable buffer and idempotent upload

Records are appended to a durable JSONL file, `trace_buffer.jsonl`, in append mode without forced flush. The buffer is capped at 8 MiB (`maxBufferBytes`); when it exceeds the cap, `_trimOldest` drops the oldest *half of records by count* and keeps the newer half (`trace_logger.dart:31,196-207`) — a lossy, count-based trim rather than a byte target, so the oldest data is silently discarded under sustained pressure.

Upload is engineered to be safely retryable. `uploadAll` atomically renames the live buffer to an immutable `trace_pending.jsonl` and writes a sidecar meta file holding a single `uploadId` and `deviceId` reused across every retry (the sidecar is regenerated only if absent), so a network failure followed by a retry re-sends the *same* `uploadId` rather than minting a new one (`trace_logger.dart:88-115`). The batch is assembled into an envelope, gzip-compressed, and POSTed to `<base>/v1/traces` with `Authorization: Bearer <token>`, `Content-Type: application/json`, and `Content-Encoding: gzip` (`trace_logger.dart:116-128`). Any HTTP 2xx deletes the pending and meta files; any non-2xx or exception keeps the pending batch for a later retry (`trace_logger.dart:130-142`). The daily-upload prompt in `main.dart` gates uploads on consent, a configured server URL and token, a new calendar day, and the presence of unuploaded data (`lib/main.dart:638-649,676-679`).

The server side (`trace_server/server.py`) mirrors this contract. It enforces a 16 MiB raw-body cap (413 before decompression), decompresses gzip, rejects a decompressed body over 4× that cap as a zip-bomb guard, validates only the envelope via a permissive pydantic model (record bodies are free-form `List[Dict]`), and honours idempotency: a known `uploadId` returns

`200 {status: duplicate, stored: 0}` without re-storing, while a fresh batch returns `201` and is written both to a per-day NDJSON archive and a WAL-mode SQLite index (`server.py:196-321`). Auth uses a constant-time Bearer comparison, and the server refuses to start without a token unless `ALLOW_NO_AUTH=1`.

1.9.6 Documentation drift uncovered

In reconciling the code against `trace_server/schema.md` and `trace_server/README.md`, several drifts surfaced and are reported here for honesty rather than papered over:

- **Identity scheme.** `schema.md` presents `deviceId` as an opaque salted/hashed pseudonym (an `h:` prefix), but the shipped code and the project's locked decision use a fresh rotating UUID v4 per upload with no salt or hash. The `schema.md` example is stale.
- **Foreground vs. background sampling.** `README.md` claims background coarse-GPS sampling populates the visit record, whereas `schema.md` correctly notes that sampling is foreground-only today with background sampling a follow-up; the code has no background-service wiring, matching `schema.md`.
- **Omitted `appVersion`.** Both `schema.md` and the server model list `appVersion`, but `TraceLogger._buildEnvelope` never sets it (`trace_logger.dart:171-179`), so it is always absent in practice.
- **Stale delivery model.** The `trace_server/README.md` "Decisions (locked)" still asserts that relaying/multi-hop is not built and that `deliveryMethod`/hop counts are logged constants — directly contradicting both the shipped mesh (see the *Mesh Routing* section) and `schema.md`, which now marks `relayHops` and `deliveryMethod` as genuinely varying measurements. The README is stale relative to the mesh migration.

Finally, two correctness caveats worth recording: `visit` tracking state (`returnTimeMs`, `visitCount`) is in-memory only and resets on process restart, so those accumulated fields are not durable despite `schema.md`'s framing; and `returnTimeMs` as computed is the gap *away* from a place (arrival minus previous departure), which differs from `schema.md`'s "since the previous visit" phrasing.

—

Files backing this section (authoritative): '/private/tmp/claude-502/-Users-dbachar-git-technion-grassroots-networking–claude-worktrees-zealous-noether-ff8f0f/eccd4b29-2dc7-4133-b2df-5aefb6578891/scratchpad/arch/sheets/12-trace.md', '/private/tmp/claude-502/-Users-dbachar-git-technion-grassroots-networking–claude-worktrees-zealous-noether-ff8f0f/eccd4b29-2dc7-4133-b2df-5aefb6578891/scratchpad/arch/sheets/13-ui-main.md', and '/private/tmp/claude-502/-Users-dbachar-git-technion-grassroots-networking–claude-worktrees-zealous-noether-ff8f0f/eccd4b29-2dc7-4133-b2df-5aefb6578891/scratchpad/arch/verified-by-main.md'.

1.10 End-to-End Interaction Walkthroughs

The preceding sections dissected the transport component by component. This section reassembles them into seven concrete narratives that trace a single event as it flows across the layers. Each walkthrough is deliberately step-by-step and cites the responsible component by file and symbol; the reader should treat these as the "integration tests in prose" that show how the sender-anonymous envelope, the Noise session layer, the managed-flooding mesh, and the Redux projection actually cooperate at run time.

1.10.1 Sending a Message

A host application calls `GrassrootsNetwork.send(recipientPubkey, payload)` (`lib/src/grassroots_network.dart`). The coordinator drives the following sequence.

1. *Validate and identify.* `send` rejects any recipient public key that is not exactly 32 bytes, returning `null`. Otherwise it mints a v4 UUID as the `messageId` (if the caller did not supply one) and sets the wire `packetId` equal to it, so that the recipient's ACK — which echoes the `packetId` — correlates back to this send.

2. *Choose a transport, BLE first.* `_trySendMessageNow` attempts the BLE mesh first and only falls back to the Internet transport when BLE is unavailable, honouring the "BLE preferred" rule.

3. *Ensure a Noise session.* The BLE send path is gated on holding a Noise session with the recipient. `_ensureNoiseSession` completes an XX handshake only for a *direct* neighbour (see below), but an *already-established* session lets even a non-adjacent recipient be addressed, because sessions are keyed by peer identity, not by path.

4. *Seal the payload.* The message is wrapped in a `secure` packet whose plaintext is a `SecureFrame` (`lib/src/models/secure_frame.dart`) carrying `contentType = message`; `NoiseSessionManager.encryptPacket` seals it to the recipient's session. The sender-anonymous envelope carries *no* wire signature — authentication is end-to-end inside Noise.

5. *Flood or send direct.* Over BLE, `_floodViaBle` broadcasts the sealed bytes to *all* neighbours and reports success when the neighbour count is greater than zero — this is managed flooding, not a point-to-point send. Over the Internet, the copy is a *direct* point-to-point UDX write: an existing connection is reused, else the coordinator connects on demand to a known address, else it discovers the peer through a facilitator (walkthrough 5).

6. *Arm the ACK watchdog.* A successful send registers the message in `_ackPendingMessages` with a `Timer` of `config.ackTimeout` (default 5 s) calling `_onAckTimedOut`. On timeout the message is re-queued onto the per-recipient outbound FIFO; an inbound ACK dispatches `MessageDeliveredAction` and clears the watchdog. A BLE path drop re-queues in-flight messages on that device immediately, without waiting for the timeout.

7. *Queue when offline.* If no transport is usable, `send` still returns the `messageId` and enqueues the payload; the periodic announce timer later drains the queue when a peer reappears.

Payloads larger than the fragmentation threshold (320 bytes) are split into 270-byte chunks by `FragmentHandler`, each carried as its own `SecureFrame` (with `fragIndex`/`fragCount`), sealed individually, flooded, and separated by a 20 ms inter-fragment delay. The whole message shares one `messageId`; reassembly is post-decrypt and keyed on it. Fragmented BLE sends require a *pre-existing* session and do not themselves trigger a handshake.

1.10.2 Receiving, Relaying, and DTN-Caching a Packet

Every inbound packet, from any transport, enters `MessageRouter.processPacket` (`lib/src/routing/message_router.dart`).

1. *Early types.* `announce` and `noiseHandshake` packets are handled and returned first — never deduplicated, never relayed.

2. *Deduplicate.* For a `secure` packet, the router calls `BloomFilter.checkAndAdd` on the outer `packetId`; `firstSeen` is the negation of prior presence. This loop/relay dedup is on the *outer* `packetId`.

3. *Is it for us?* `_isForUs` compares the envelope's `recipientPubkey` to our identity.

4. *If for us: trial-decrypt and deliver.* A session-encrypted packet addressed to us is authenticated by trial-decrypting it against every active Noise session; the AEAD tag identifies the right one and recovers the sender public key. The router then `SecureFrame.decode`s the plaintext and dispatches on `frame.contentType` (message, ack, readReceipt, signaling). Delivery to the application happens only on `firstSeen`; a duplicate is re-ACKed without re-delivering. *Delivery* dedup is thus on the *inner* `messageId`, distinct from the outer-`packetId` relay dedup. A cleartext packet addressed to us is dropped as unauthenticated.

5. *If not for us: relay.* When `firstSeen && ttl > 1` and the per-neighbour budget (at most 512 relays per 10 s per inbound neighbour) allows it, `onRelay` re-floods `packet.decrementTtl()` to all BLE neighbours except the inbound path. A packet at `ttl <= 1` dies at the last hop. Relaying is BLE-only; the Internet path never relays.

6. *DTN cache on miss.* If, after the relay decision, the recipient is *not* currently reachable, the sealed packet is cached in the bounded `DtnStore` (256 recipients, 32 packets each, 6 h max age) keyed by recipient hex — store-carry-forward for a peer that is out of range.

7. *Flush on reappearance.* When that recipient later reappears, the coordinator calls `flushDtnFor`, which takes all non-expired cached packets for the recipient and re-floods each via `onRelay`.

Because a relay only ever sees the recipient ID and opaque sealed bytes, it can neither read nor authenticate the traffic it carries — the accepted cost of sender-anonymity, bounded by TTL, dedup, and the per-neighbour rate cap.

1.10.3 Discovery, Dual-Role Convergence, and ANNOUNCE

Presence over BLE is established locally, never flooded.

1. *Advertise and scan.* Each device advertises a pubkey-derived service UUID whose fixed `grassroots` prefix (`84c403160871e5ad`) is followed by a suffix that rotates every 15 minutes (`SHA-256("grassroots ble suffix" | pubkey | slot)`), and scans continuously for that prefix (`lib/src/transport/ble_transport_service.dart`). The UUID is only a discovery hint — it rotates for unlinkability and proves nothing; a peer recognises it by recomputing the current and adjacent slots.

2. *Converge to dual role.* The pair arbitrates who dials which leg from advertisement markers: same-platform peers use the lower service UUID as the deterministic first-mover; iOS advertises `grs-ios` and owns the first leg toward an Android because a measured iOS constraint forbids it opening the *second* link. Both legs are pursued until two GATT connections exist; a single link is only ever a transient state that the transport keeps trying to upgrade.

3. *Self-signed ANNOUNCE.* Over the connection each side sends an ANNOUNCE built with TTL 1 whose payload carries the full public key, nickname, and an Ed25519 signature over the payload (`createAnnouncePayload`). ANNOUNCE is the *one* signed, non-relayed packet type.

4. *Verify and gate.* `MessageRouter` verifies the ANNOUNCE via `protocolHandler.decodeAnnounce`, which throws on a forged or malformed signature; the router drops it. The coordinator then applies the cold-call gate: in `ColdCallTrustLevel.open` any sender is accepted, otherwise only accepted-friend public keys are; a rejected ANNOUNCE triggers a BLE disconnect of the device.

5. *Project to Redux.* On first receipt per identity, `onPeerDiscovered` fires (deduped), and the verified identity is recorded in the store. Note that discovery precedes and is decoupled from the Noise-gated `onPeerConnected`.

Crucially, ANNOUNCE is *neighbour-local*: it is not relayed (TTL 1), so a node learns the presence of *direct* neighbours only. A non-adjacent peer's presence is never learned as a discovery event; the mesh reaches such a peer only implicitly, by flooding sealed traffic addressed to its recipient ID (walkthrough 2), not by propagating its ANNOUNCE.

1.10.4 Noise XX Session Establishment

A session is built by a neighbour-local three-message XX handshake (`Noise_XX_25519_ChaChaPoly_SHA256`, `lib/src/session/noise_session_manager.dart`).

1. *Initiate.* `_ensureNoiseSession` returns early if a session already exists; otherwise `startHandshake` produces message 1 (a 32-byte ephemeral X25519 key), packaged in a `noiseHandshake` packet with TTL 1 — handshakes are not relayed through the mesh.

2. *Respond.* The peer's `handleHandshakePacket` replies with message 2 (80 bytes: ephemeral plus encrypted static), and the initiator replies with message 3 (48 bytes: encrypted static). Because the envelope is sender-anonymous, each side resolves the remote public key from the inbound *transport path* (BLE/UDP `getPubkeyForPeerId`), not from any wire field.

3. *Verify identity.* The Noise-delivered remote static is checked, in constant time, to equal `pkToCurve25519(remotePubkey)` — the libsodium birational map binding the Ed25519 identity (conveyed inside the encrypted handshake) to the X25519 static.

4. *Break collisions.* If both peers initiate at once, the tie is broken by hex comparison: the lower public key keeps its initiator role and ignores the inbound message 1; the higher abandons and becomes responder.

5. *Establish and drain.* On completion `split` derives the directional transport keys and `_onNoiseSessionEstablished` dispatches the authenticated-connection actions and drains the queued outbound messages. The session is kept across path changes and even across `stop()`, because it is end-to-end and path-independent.

From this point, application AEAD binds an 81-byte AAD (the constant `secure` wire type, sender, recipient, `packetId`) but deliberately *excludes* TTL and timestamp, the fields a relay mutates. A from-scratch multi-hop IK/KK handshake that would let two non-adjacent peers negotiate a session through relays is **future work**; today handshakes are strictly neighbour-local.

1.10.5 NAT Reconnection via a Facilitator

Two NAT'd friends who have no live path re-establish a *direct* Internet connection through a mutually trusted facilitator — a configured rendezvous server or a well-connected friend — which relays only signalling metadata and then leaves the data path (`lib/src/signaling/signaling_service.dart` plus the coordinator).

1. *Register.* Each device periodically re-registers its NAT-observed address with its facilitators; a facilitator records the address and reflects the observed external address back via `ADDR_REFLECT` (the STUN-equivalent).

2. *Request a rendezvous.* To reach an offline friend, the initiator fans out a `RECONNECT` (or `AVAILABLE` to the target's known rendezvous servers) to trusted facilitators. All signalling rides inside a sealed `secure/signaling SecureFrame`, so the facilitator authenticates the sender by trial-decrypt — UDP signalling is friend-only.

3. *Coordinate the punch.* When the facilitator has both sides (matched within a 15 s coordination cooldown, constrained to a shared address family), it sends each peer a `PUNCH_INITIATE` carrying the *other* side's ip/port.

4. *Punch.* Each side sprays 36-byte "BCPU" punch packets at the other (`HolePunchService`), opening its own NAT mapping; the initiator is chosen deterministically as the lexicographically smaller public key and, on receiving `PUNCH_READY` from the counterpart, proceeds to connect, while the non-initiator sustains punch traffic and waits.

5. *Connect direct.* The initiator opens the UDX connection by sending a signed ANNOUNCE to the punched target (with `performPreConnectPunch: false`). From here the two peers talk directly; the facilitator is no longer in the path.

Two caveats keep this narrative honest against the code. The `HolePunchService` itself only sprays bursts at a fixed duration/interval; the deterministic-initiator choice and `PUNCH_INITIATE/PUNCH_READY` coordination live in the coordinator and `SignalingService`, not in the punch service. And rendezvous servers are the *current*, fully-implemented NAT-bootstrap mechanism; replacing them entirely with well-connected friends (and signed invite links) is a design direction, **not** yet shipped.

1.10.6 Trace Capture and Daily Upload

When the user has granted consent (`SettingsState.traceLoggingConsent`), the transport records anonymised operational telemetry (`lib/src/trace/trace_logger.dart`).

1. *Capture.* Mesh-aware hooks call `TraceLogger.log` with per-event records — `message` (with `dir` of sent, recv, delivered, ack_timeout, or dup; `recv` carries `relayHops = defaultTtl - ttl` and a `direct`/`relayed` `deliveryMethod`), `contact`, `density`, `visit`, `device`, and `buffer` (reporting `outboundQueued` and `dtnBuffered` occupancy). `log` is a no-op when disabled and never throws; records append to an 8 MiB JSONL buffer whose oldest half is dropped on overflow.

2. *Sample coarse context.* A periodic `LocationSampler` emits a coarse geohash (precision 6) density fix and a `visit` record on cell change; continuous *background* sampling is a follow-up, not yet wired.

3. *Anonymise and pack.* On upload, each distinct peer public key is replaced by a per-upload alias (`p0`, `p1`, ...) and a fresh per-upload `deviceId` UUID is minted, making uploads unlinkable. The envelope is gzip-compressed.

4. *Upload idempotently.* The live buffer is renamed to an immutable pending file with a sidecar carrying a stable `uploadId`, then POSTed to `/v1/traces` with a Bearer token. A 2xx deletes the pending files; any failure keeps them for retry, and the server treats a repeated `uploadId` as a duplicate — so a retried daily upload never double-stores.

1.10.7 Transport Event to Redux to UI

The store is a strict projection of transport facts, never an inference, and the UI reads only from it.

1. *Transport emits a fact.* A concrete transport event — an authenticated Noise session established, a UDX path torn down, a BLE device disconnected — is surfaced explicitly. UDP disconnect is *immediate*: the coordinator dispatches `PeerUdpConnectionChangedAction(connected: false)` rather than inferring loss from silence.

2. *Reducer updates a slice.* The action updates the relevant `PeersState`/`TransportsState` fields. Consolidated reachability is gated on an authenticated session: a session established dispatches `PeerBleAuthenticatedAction` or `PeerUdpConnectionChangedAction(connected: true)`. (A rendezvous server is the deliberate exception — its raw UDX connection counts as reachable without any Noise session.)

3. *Reachability transition.* `processReachabilityTransitions` diffs each peer's `isReachable` against the previous tick and fires `onPeerConnected`/`onPeerDisconnected` *only* on a change. Because reachability is consolidated end-to-end, losing one of two live transports fires nothing; the callback fires only on the transition to or from zero live transports. This is what makes the two transports independent: a BLE drop never degrades a UDP-derived online status, and the end-to-end Noise session is kept regardless.

4. *UI reacts.* The UI subscribes to the store and re-renders on the changed slice, showing the peer's online state and message-delivery status without ever consulting a transport directly.

1.11 Design Decisions, Trade-offs & Future Work

This section steps back from the mechanics of the individual layers and examines the load-bearing decisions that shape the transport as a whole. Each is a deliberate trade-off, and several of them cost something real; the point of collecting them here is to make those costs explicit and to separate what the code *does today* from what the design *intends* but has not yet built.

1.11.1 Sender-anonymous open relay versus authentication

The single most consequential decision is that the BLE mesh is an *open, sender-anonymous relay*. A node rebroadcasts any packet it receives toward the recipient, decrementing TTL and dropping at zero, and it does so for recipients with which it has no relationship whatsoever (`lib/src/routing/message_router.dart`). This is what lets a message reach a friend who is not a direct neighbour, and it is a deliberate reversal of the older "never relay for arbitrary peers" posture.

The enabling design choice is that the outer envelope carries *only* the recipient ID — there is no sender field and no whole-packet Ed25519 signature (`lib/src/models/packet.dart`). A relay therefore cannot tell who originated a packet, which is exactly the property that makes open relaying tolerable from a privacy standpoint: an intermediary forwards opaque, recipient-addressed bytes it can neither read nor attribute.

The cost is equally direct: because the envelope has no sender, **a relay cannot authenticate what it forwards**. There is no hop-by-hop signature check; relaying is unverified by construction. Authentication is recovered only *end-to-end*, inside Noise: the recipient trial-decrypts an inbound `secure` packet against its active sessions, and the AEAD tag both selects the right session

and proves integrity (`lib/src/session/noise_session_manager.dart`, `trialDecrypt`). The peer's Ed25519 identity is conveyed inside the encrypted handshake and bound to the Noise static key via the birational map — never in a cleartext header. This is discussed in the *Identity, Trust & the Security Model* and *Noise Session Layer* sections; what matters here is the trade-off it forces.

Since unverified relaying cannot be bounded by cryptography, it is bounded by *resource limits* instead. Three mechanisms cap abuse: a TTL that strictly decrements and drops the packet at ttl ≤ 1; a `BloomFilter` of seen packet IDs that suppresses loops and re-sends; and a per-neighbour relay budget of `_maxRelaysPerWindow` = 512 packets per `_relayWindow` = 10 s (`lib/src/routing/message_router.dart`). An honest but important asymmetry: the per-neighbour rate limit applies *only* to attributable BLE neighbours. When the inbound peer is unattributable — the UDP case, where `inboundPeerId == null` — `_allowRelayFrom` returns `true` unconditionally, so UDP is *not* rate-limited. In practice the Internet transport is direct point-to-point and rarely relays, so this branch seldom fires, but it is a genuine gap rather than a designed safeguard.

1.11.2 Store-carry-forward bounding

Store-carry-forward (DTN) lets a relay cache a packet whose recipient is currently out of range and re-flood it when that recipient reappears. Left unbounded this is an obvious battery sink and abuse vector, so every dimension of the cache is capped: `DtnStore` holds at most `maxRecipients` = 256 recipients, `maxPerRecipient` = 32 packets each, expiring after `maxAge` = 6 hours (`lib/src/mesh/dtn_store.dart`). Caching happens only inside the relay branch after a successful relay decision; if the per-neighbour budget already blocked the relay, the packet is not cached either. The sender additionally keeps its *own* outbound queue and re-floods its un-acked messages independently of any relay's cache — these are two distinct bounded buffers, surfaced separately as `outboundQueuedCount` and `dtnBufferedCount`.

1.11.3 Dual-role BLE and transport independence

Two structural decisions are treated as inviolable. First, every BLE pair must converge to a *dual-role* connection — two GATT legs, each device central on one and peripheral on the other — with platform asymmetries solved by choosing *who initiates each leg* rather than by abandoning a leg; a single-link pair is acceptable only as a transient state the transport keeps trying to upgrade. Second, BLE and UDP are *independent*: disabling or losing one has zero effect on the other's reachability, and application-level connect/disconnect callbacks report consolidated end-to-end reachability, firing only on the transition to or from zero live transports. Both are elaborated in the transport sections; they are noted here because they are constraints the design refuses to trade away.

1.11.4 Unlinkable BLE beacon versus reconnection churn

The BLE service UUID keeps a fixed Grassroots prefix but rotates its per-pubkey suffix every fifteen minutes, deriving the suffix as SHA-256("grassroots ble suffix" $\|$ pubkey $\|$ slot)[: 8] over a wall-clock slot of `bleSlotDuration` = 15 min (`lib/src/models/identity.dart`). The payoff is unlinkability: a passive observer who does not hold the public key sees a beacon that changes every slot and cannot use it as a stable tracking handle, while a friend — or the device itself — recomputes the current and adjacent slot suffixes (`candidateServiceUuids`, `serviceUuidMatchesPubkey`) and still recognises the peer. The slot period is chosen to match the roughly fifteen-minute cadence of BLE MAC randomisation, so the advertised UUID and the radio address rotate together rather than one outliving the other as a correlator.

The cost is real and paid on a timer. The BLE transport runs a thirty-second slot-check (`_maybeReAdvertiseForSlot`) and, on each boundary crossing, tears down and rebuilds the GATT advertisement under the new suffix. That rebuild is the accepted price of an unlinkable beacon: a pair that happens to be mid-handshake or mid-message across the boundary can see its advertisement — and, on some stacks, its GATT server — briefly drop and reform, adding reconnection churn every quarter hour. The adjacent-slot recognition window (previous, current, next) exists precisely to keep unsynchronised clocks and boundary races from breaking friend recognition during that reform, but the reconnect itself is not free. The design accepts periodic churn in exchange for the privacy property; the alternative — a fixed per-pubkey UUID that never rotates — would remove the churn but hand any passive scanner a permanent device identifier, which the project treats as the worse trade.

1.11.5 Wire-type tightening: collapsing eighteen packet types to three

A recently shipped refactor collapsed the wire `PacketType` from eighteen values to exactly three: `announce`, `noiseHandshake`, and `secure` (`lib/src/models/packet.dart`). Everything that is not an announce or a handshake is now one opaque `secure` envelope. The content type (message, ack, read-receipt, signaling) and any fragmentation moved *inside* the sealed payload as a `SecureFrame` — a 21-byte inner header `[contentType:1][fragIndex:2][fragCount:2][messageId:16][chunk...]` (`lib/src/models/secure_frame.dart`).

The motivation is a closed metadata leak. Previously a relay reading a header could distinguish `secureAck` from `secureMessage` from `secureSignaling` from the fragment variants, learning the content class and fragment structure of traffic it merely forwarded. Now a relay sees only `secure` (`0x03`) for all content and cannot tell an ack from a message from a fragment. This is the same class of leak the handshake security audit flagged. The content type remains authenticated because it lives inside the AEAD-protected plaintext, and the application AAD binds the constant `secure` wire type plus sender, recipient, and packet ID while excluding TTL and timestamp — the fields a relay mutates (`lib/src/session/noise_session_manager.dart`). Fragmentation is now reassembled *post-decrypt*, keyed by the inner `messageId`, while relay/loop dedup stays on the outer `packetId` and delivery dedup on the inner `messageId`. The now-dead `nack` type was removed.

The honest caveat: this is shipped **in the client only** (`flutter test` 589/589). The `bootstrap_anchor` mirror still uses the old eighteen-type wire, so **client ↔ anchor interop is temporarily broken** until that follow-up lands. Mirroring the anchor is a committed, near-term task, not open-ended future work.

1.11.6 Clean breaks — and the honest exceptions

The project's stated rule is clean breaks with no legacy or compatibility shims: wire decoders throw on malformed input rather than tolerating a hypothetical old version, and WebRTC — once a vestigial `TransportType` enum member with no implementation — has now been removed entirely, leaving `TransportType` as just `ble, udp` (`lib/src/transport/transport_service.dart`). Accuracy requires naming where reality diverges from the rule:

- `Block.tryDeserialize` (`lib/src/models/block.dart`) still tolerates "legacy plain text" by returning `null` for input that is not a well-formed block — a compatibility-flavoured probe that sits uneasily beside the no-legacy directive.
- Several stale comments contradict the current design: the `BleTransportService` class doc still claims "Direct delivery only. No relaying, no store-and-forward," and numerous

comments across the coordinator, UDP transport, and UI still reference "rendezvous servers" that the design note proposes to remove.

- Some Redux `hashCode` implementations are intentionally weak — `MessagesState`, `FriendshipsState`, and `SignalingState` hash only collection *lengths*, so structurally different states of equal size collide (equality remains content-exact). This is a deliberate performance shortcut worth flagging rather than a defect.

1.11.7 Known limitations and future work

The following are **not yet implemented**; they are labelled as such deliberately to avoid presenting intent as fact.

- **From-scratch multi-hop session establishment (IK/KK).** Today the Noise handshake is neighbour-local: both `announce` and `noiseHandshake` packets are sent with TTL 1 and are never relayed (`lib/src/grassroots_network.dart`). A message can flood multiple hops *only after* a session already exists, because the flooded XX handshake cannot address its reply back along the mesh. A redesign toward Noise IK/KK is planned to close this gap. The handshake security audit attaches must-do conditions to any such change: gate the IK path behind the cold-call/friend policy, carry *no* application data on IK message 1, guard against eviction of a live session by an unauthenticated handshake attempt, and use distinct protocol names per pattern to prevent cross-pattern confusion.
- **Rendezvous-server removal.** Rendezvous servers are *still fully implemented and in use* — the anchor/RV plumbing, `RendezvousServerSettings`, `RV_LIST`, and `bootstrap_anchor` are all present and active, and RV is the current NAT-bootstrap mechanism. The commit proposing to replace them with well-connected friends plus signed invite links (`grassroots://` deep links) is a *design note only*; none of the invite-link build wiring (Android intent filters, iOS URL schemes, an `app_links`-style dependency) yet exists.
- **Hole-punch coordination.** `HolePunchService` only sprays punch bursts at a fixed duration and interval; it has no notion of a deterministic initiator, no `PUNCH_INITIATE`, and no friend-gating. The deterministic-initiator selection and simultaneous-punch orchestration live partly in the coordinator and `SignalingService`, so the idealised "friend coordinates a simultaneous hole-punch" flow is only partially realised in code.
- **Deferred trace fields.** The trace-logging envelope omits `appVersion` (never set by `_buildEnvelope` despite the server accepting it), continuous background geolocation for visit records is a follow-up rather than shipped, and in-memory visit-tracking state resets on restart — so counts framed as accumulated "to date" are not durable.
- **Bounded replay-window edge.** The Noise replay guard is a set capped at 2048 nonces with FIFO eviction (`lib/src/session/noise_session_manager.dart`). It is a *bounded*, not absolute, guard: a nonce evicted after 2048 newer nonces have been seen could in principle be replayed. This is an accepted memory/security trade-off rather than a full sliding-window replay defence.